

Appunti di Algorithm Engineering

Anno Accademico 2026/2027

Indice

1	Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria	1
1.1	Definizione Formale di un Algoritmo	1
1.1.1	Algoritmi Esatti, Intelligenza Artificiale ed Euristiche	2
1.2	Misurazione dell'Efficienza e Analisi Asintotica	3
1.2.1	Perché Non Contare il Numero Esatto di Istruzioni?	3
1.2.2	Perché Preferire l'Analisi al Caso Pessimo?	4
1.3	Modelli Computazionali: RAM e Hardware Reale	4
1.3.1	Il Modello Random Access Machine	4
1.3.2	Hardware Reale: Gerarchia di Memoria e Memory Wall	5
1.3.3	Quando il Modello RAM Non È Sufficiente	7
1.4	Località e Modello a Due Livelli	7
1.4.1	Il Modello di Memoria a Due Livelli	8
1.5	Esempio Pratico: Scansione di un Array Lineare	9
1.5.1	Analisi nel Classico Modello RAM	9
1.5.2	Analisi nel Modello a Due Livelli	9
1.5.3	Accesso Lineare contro Accesso Sparso	10
2	Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale	11
2.1	Accesso Non Sequenziale: L'Algoritmo $A_{s,b}$	11
2.1.1	Condizione di Coprimalità	12
2.1.2	Costo di I/O	12
2.2	Limiti Algoritmici e Accelerazione Hardware	13
2.3	Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree	14
2.3.1	Ricerca Binaria in Memoria Secondaria	14
2.3.2	B-Tree e B ⁺ -Tree	15
2.3.3	Confronto Quantitativo	16
2.4	Algoritmi I/O-Aware e Paginazione Virtuale	16
2.4.1	Paginazione con un Working Set Sovradimensionato	17
2.4.2	Esempio Numerico di Thrashing	17
2.5	Punti Chiave	18

3	Permutazione, Ordinamento e Multiway Merge Sort in Memoria Esterna	19
3.1	Il Problema della Permutazione vs. Ordinamento	19
3.1.1	Formulazione del Problema	19
3.1.2	Implementazione Ingenua su Modello RAM	21
3.1.3	Principio di Simulazione (RAM contro Due Livelli)	21
3.1.4	La Relazione tra Ordinamento e Permutazione	22
3.2	Complessità di Permutazione in Memoria Esterna	25
3.2.1	Analisi dei Casi Limite	25
3.2.2	Verifica Numerica su Parametri Architettureali Reali	25
3.3	Binary Merge Sort in Memoria Esterna	26
3.3.1	Architettura dei Buffer nella Procedura di Fusione	27
3.3.2	Complessità dell'Albero di Ricorsione	28
3.3.3	Criticità e Limiti di Efficienza	29
3.4	Equivalenza Asintotica del Costo di Ordinamento Esterno	30
3.5	Multiway Merge Sort (k -Way Merge Sort)	30
3.5.1	Configurazione e Architettura della Memoria Interna	31
3.5.2	Dinamica dell'Algoritmo ed Evoluzione dei Livelli	32
3.6	Ottimizzazione dei Run Iniziali: Lo Spazzaneve (<i>Snow Plow</i>)	33
3.6.1	Meccanismo Operativo della Replacement Selection	34
3.6.2	Esempio Passo-Passo dell'Algoritmo	35
3.6.3	Teorema della Lunghezza Media dei Run	36
3.7	Parallelismo su Più Dischi: <i>Disk Striping</i>	37
3.7.1	Lower Bound di Ordinamento Parallelo	38
3.7.2	L'Architettura del Disk Striping	38
3.7.3	Analisi del Rallentamento (<i>Slowdown</i>)	39
3.7.4	Discussione dei Regimi Asintotici e Limiti Fisici	40
4	Limiti Inferiori per l'Ordinamento, Quicksort Dual-Pivot e Bounded Quicksort	41
4.1	Modello RAM: Limiti Inferiori e Alberi di Decisione	42
4.1.1	Il Modello dell'Albero di Decisione	42
4.1.2	Il Teorema Generale dell'Informazione	43
4.1.3	Applicazioni Fondamentali: Ricerca e Ordinamento	44
4.2	Modello a Due Livelli: Limite Inferiore di Aggarwal-Vitter	45
4.2.1	La Natura dell'I/O e i Confronti Gratuiti in Memoria Interna	46
4.2.2	Calcolo del Grado di Diramazione (Fan-out) dell'Albero di I/O	46
4.2.3	Derivazione Formale del Lower Bound di Aggarwal-Vitter	48
4.3	Quicksort: Gestione dei Duplicati e Dual-Pivot Quicksort	50

4.3.1	Il Problema dei Valori Uguali (Equal Values)	50
4.3.2	Algoritmo di Partizionamento a 3 Vie ed Esempio Esecutivo	51
4.3.3	Architettura del Dual-Pivot Quicksort (Yaroslavskiy) .	54
4.3.4	Invariante e Logica di Avanzamento	55
4.3.5	Esempio Esecutivo e Traccia del Dual-Pivot Quicksort	58
4.4	Bounded Quicksort (BQS) ed Eliminazione della Tail Recursion	59
4.4.1	La Criticità dello Stack di Ricorsione	60
4.4.2	Algoritmo Bounded Quicksort (BQS)	60
4.4.3	Esempio Esecutivo e Gestione dello Stack nel Bounded Quicksort	62
4.4.4	Analisi Rigorosa dello Spazio Extra $\Theta(\log_2 n)$	64
4.4.5	Ottimizzazione Ibrida con Cutoff a Insertion Sort . . .	65

Capitolo 1

Progettazione di Algoritmi, Modelli Computazionali e Gerarchia di Memoria

Lezione 1

Introduzione, modelli computazionali e gerarchia di memoria

Data: 15 settembre 2026

1.1 Definizione Formale di un Algoritmo

Un *algoritmo* è una sequenza finita e ordinata di istruzioni elementari non ambigue che, a partire da un insieme specificato di dati di input, produce un corrispondente insieme di dati di output in un lasso di tempo finito.

Nella formulazione classica presentata in *The Art of Computer Programming* [1], un algoritmo deve soddisfare cinque proprietà fondamentali:

Finitudine. L'algoritmo deve sempre terminare dopo un numero finito di passi di calcolo per ogni configurazione di input ammissibile.

Definitezza. Ogni istruzione deve essere formulata in modo chiaro, preciso e senza ambiguità; lo stato di esecuzione successivo deve essere univocamente determinato.

Input. L'algoritmo riceve zero o più quantità specificate dall'esterno prima che l'esecuzione abbia inizio.

Output. L'algoritmo produce una o più quantità di output che hanno una relazione specificata con i dati di input. L'output deve essere significativo per il problema che si sta risolvendo.

Efficacia. Ogni operazione prescritta deve essere elementare e fisicamente eseguibile in tempo finito utilizzando una quantità limitata di risorse, come carta e penna o una macchina sequenziale.

1.1.1 Algoritmi Esatti, Intelligenza Artificiale ed Euristiche

Un classico algoritmo esatto si distingue da un metodo di intelligenza artificiale o euristico per le garanzie che fornisce e per il modo in cui esplora lo spazio delle decisioni.

Tabella 1.1: Confronto tra algoritmi esatti, intelligenza artificiale e metodi euristici.

Dimensione	Algoritmo esatto classico	Metodo di IA o euristico
Garanzia di correttezza	Produce una soluzione corretta per ogni istanza ammissibile; l'ottimalità è garantita quando la specifica del problema richiede un ottimo.	Può privilegiare la qualità empirica o l'efficienza pratica e può mancare di una garanzia universale per ogni istanza.
Terminazione	Può essere dimostrata in anticipo utilizzando funzioni potenziale o varianti di ciclo.	Spesso legata a criteri di arresto euristici, come un numero massimo di epoche o la convergenza del gradiente.
Spazio delle decisioni	Determinato da una procedura specificata in modo preciso, che può essere deterministica o randomizzata.	Esplorato tramite ricerca euristica, inferenza statistica o ottimizzazione di una funzione di perdita.

La *programmazione euristica* non cerca una dimostrazione formale di ottimalità assoluta su ogni istanza ammissibile. Piuttosto, cerca un compromesso efficiente per problemi computazionalmente intrattabili, come i problemi NP-completi o NP-ardui.

1.2 Misurazione dell'Efficienza e Analisi Asintotica

1.2.1 Perché Non Contare il Numero Esatto di Istruzioni?

Definire il costo esatto di esecuzione $T_A(n)$ come il numero effettivo di cicli di clock o di istruzioni elementari presenta diverse difficoltà:

1. **Dipendenza dall'architettura:** il risultato dipenderebbe dalla specifica CPU, dall'architettura del set di istruzioni (ISA) e dalla frequenza di clock.
2. **Dipendenza dalla toolchain:** varierebbe a seconda del compilatore, del livello di ottimizzazione e del sistema operativo.
3. **Mancanza di generalità:** oscurerebbe la qualità intrinseca dell'idea algoritmica.

Per questo motivo, l'analisi degli algoritmi utilizza la notazione asintotica. Essa astrae dai dettagli implementativi e dalle costanti moltiplicative, e studia il tasso di crescita della funzione di costo al tendere della dimensione dell'input all'infinito: $n \rightarrow \infty$.

O-grande (limite superiore asintotico):

$$f(n) = \mathcal{O}(g(n)) \iff \exists c > 0, n_0 > 0 \text{ tali che} \\ 0 \leq f(n) \leq c g(n), \quad \forall n \geq n_0.$$

Omega-grande (limite inferiore asintotico):

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 > 0 \text{ tali che} \\ 0 \leq c g(n) \leq f(n), \quad \forall n \geq n_0.$$

Theta-grande (limite asintotico stretto):

$$f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n)) \wedge f(n) = \Omega(g(n)).$$

1.2.2 Perché Preferire l'Analisi al Caso Pessimo?

- **Garanzia assoluta:** un limite superiore garantisce che l'algoritmo non impiegherà mai più tempo del limite teorico. Questo è essenziale per i sistemi real-time e mission-critical.
- **Semplice derivazione formale:** l'analisi al caso pessimo non richiede assunzioni probabilistiche sulla distribuzione dell'input, le quali sono spesso necessarie per l'analisi al caso medio.

1.3 Modelli Computazionali: RAM e Hardware Reale

Un algoritmo non viene eseguito in un vuoto astratto: viene eseguito su una macchina con processori, memorie e meccanismi di trasferimento dati. Un modello computazionale specifica quali dettagli della macchina sono rilevanti per l'analisi e quali vengono deliberatamente ignorati. Il modello RAM è il punto di partenza standard; l'hardware reale spiega poi quando tale astrazione diviene insufficiente.

1.3.1 Il Modello Random Access Machine

Il modello Random Access Machine (RAM) è un'astrazione sequenziale, basata su parole (word), ispirata all'architettura di von Neumann. Una RAM è costituita da un processore, un programma finito e una memoria indirizzabile di parole. Le sue assunzioni abituali sono:

- **Esecuzione sequenziale:** un processore esegue un'istruzione elementare alla volta.
- **Dati della dimensione di una parola:** l'aritmetica, i confronti, le assegnazioni e il calcolo degli indirizzi su un numero costante di parole richiedono tempo costante.
- **Accesso uniforme alla memoria:** la lettura o la scrittura di qualsiasi indirizzo costa $\mathcal{O}(1)$, indipendentemente dall'indirizzo e dalla storia degli accessi.
- **Memoria sufficiente:** l'input e le strutture dati risiedono interamente nello spazio di indirizzamento modellato; il costo di allocazione dello spazio viene analizzato separatamente quando risulta rilevante.

Il modello RAM è utile perché rende confrontabili le idee algoritmiche. Ad esempio, un ciclo che esamina n elementi di un array esegue $\Theta(n)$ operazioni RAM, indipendentemente dal particolare processore utilizzato per eseguirlo. Il modello non afferma che una macchina reale possieda memoria infinita o una latenza veramente costante; esso dichiara quali costi vengono astratti.

1.3.2 Hardware Reale: Gerarchia di Memoria e Memory Wall

Su una macchina moderna, il processore è molto più veloce delle memorie che contengono grandi insiemi di dati. Questo crescente divario tra calcolo e movimento dei dati è noto come *memory wall*. Inoltre, la memoria locale non è una singola risorsa uniforme: è organizzata in livelli con capacità, latenze e granularità di trasferimento differenti.

Una gerarchia semplificata, ordinata dal livello più veloce e più piccolo a quello più lento e più grande, è:

Registri della CPU $\longrightarrow$ Cache L1 $\longrightarrow$ Cache L2 $\longrightarrow$ Cache L3 $\longrightarrow$ DRAM
 $\longrightarrow$ NVMe/SSD/disco $\longrightarrow$ Storage remoto / cloud.

I registri e le cache fanno parte del sistema di memoria veloce del processore; la DRAM è l'usuale memoria principale; gli SSD e i dischi forniscono archiviazione persistente. Lo storage remoto e i servizi cloud sono inclusi qui solo come una più ampia gerarchia di accesso ai dati, in quanto sono raggiunti attraverso una rete piuttosto che attraverso il bus di memoria locale.

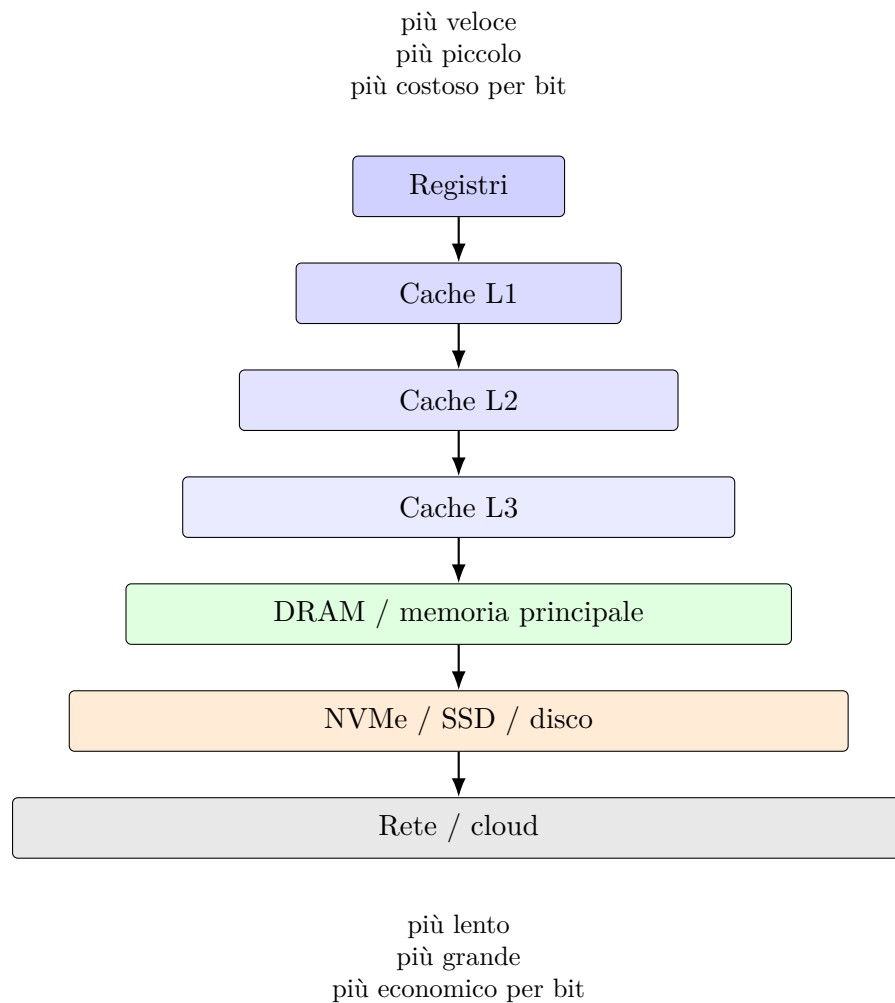


Figura 1.1: Una vista gerarchica della memoria e dell'archiviazione.

Figura 1.1 descrive i principali livelli di memoria locale: ogni strato inferiore scambia una minore velocità di accesso per una maggiore capacità.

Spostandosi verso il basso nella Figura 1.1, la capacità generalmente cresce mentre crescono anche la latenza e il costo di trasferimento. I rapporti esatti dipendono dall'hardware e dal carico di lavoro, per cui valori come da 100 a 200 volte per la DRAM rispetto alla L1, o da 10^4 a 10^6 volte per il disco rispetto alla L1, dovrebbero essere letti come esempi di ordine di grandezza piuttosto che come costanti universali.

1.3.3 Quando il Modello RAM Non È Sufficiente

Il modello RAM è una buona prima approssimazione quando il calcolo è prevalentemente sequenziale, il working set risiede interamente nella memoria principale e il costo del movimento dei dati non è il fattore dominante. Un modello diverso è preferibile quando uno dei seguenti effetti controlla le prestazioni:

Calcolo su GPU. L'esecuzione SIMT (Single Instruction, Multiple Threads) utilizza molti core di elaborazione e distingue tra spazi di memoria condivisa, locale e globale.

Memoria esterna. Nel calcolo out-of-core, il dataset non risiede interamente nella memoria veloce, per cui i trasferimenti a blocchi e le operazioni di I/O dominano il costo.

Calcolo parallelo e distribuito. Diversi processori o macchine eseguono simultaneamente e la sincronizzazione e la comunicazione diventano costi espliciti.

Informatica quantistica. Qubit, sovrapposizione, entanglement e porte unitarie richiedono un modello basato su operazioni quantistiche piuttosto che su parole RAM.

La lezione ingegneristica è quindi non di scartare il modello RAM, ma di utilizzare il modello più semplice che catturi ancora il collo di bottiglia del sistema target.

1.4 Località e Modello a Due Livelli

I sistemi moderni sfruttano due proprietà statistiche dei programmi reali:

1. **Località temporale:** se una locazione di memoria viene referenziata al tempo t , è probabile che venga referenziata nuovamente a breve, al tempo $t + \Delta t$.
2. **Località spaziale:** se una locazione di memoria x viene referenziata, locazioni vicine come $x + 1$, $x + 2$ e così via, verranno probabilmente referenziate a breve.

1.4.1 Il Modello di Memoria a Due Livelli

Quando si elaborano gigabyte o terabyte di dati, la gerarchia multilivello può essere astratta come un modello a due livelli, associato al modello a memoria esterna [2], noto anche come modello Disk Access Machine.

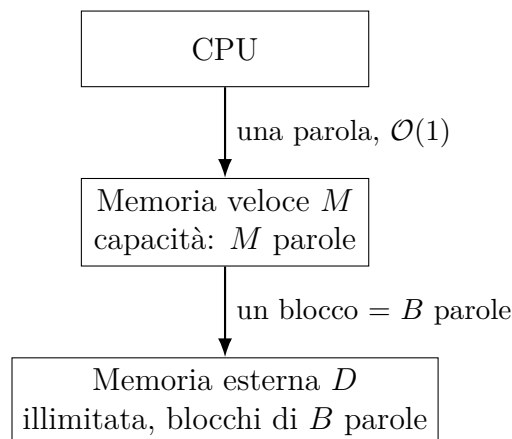


Figura 1.2: Modello di memoria a due livelli con memoria veloce M e memoria esterna D .

Figura 1.2 descrive l'astrazione a due livelli: la CPU opera su parole nella memoria veloce M , mentre i trasferimenti tra M e la memoria esterna D avvengono in blocchi di B parole.

Parametri del Modello

N : numero totale di elementi del problema da elaborare.

M : dimensione della memoria veloce interna, misurata come il numero massimo di elementi che possono essere contenuti simultaneamente.

B : dimensione del blocco trasferito in una singola operazione di I/O, con $M \geq 2B$.

Metriche di Costo

1. **Complessità di I/O (metrica primaria):** numero di trasferimenti di blocchi di dimensione B tra la memoria lenta e quella veloce.
2. **Tempo di CPU (metrica secondaria):** numero di operazioni logiche e aritmetiche eseguite dopo che i dati sono stati caricati in M .

3. **Spazio:** numero di blocchi allocati sulla memoria secondaria.

Questa astrazione è frattale: può modellare sia i trasferimenti RAM–disco sia i trasferimenti cache L3–RAM, ridefinendo il significato di M e B .

1.5 Esempio Pratico: Scansione di un Array Lineare

Consideriamo un array contiguo di N interi scansionato in sequenza, ad esempio per calcolare la somma $\sum_{i=1}^N A[i]$.

Listing 1.1: Scansione sequenziale di un array

```
1 long sum = 0;
2 for (int i = 0; i < N; i++) {
3     sum += A[i];
4 }
```

1.5.1 Analisi nel Classico Modello RAM

- Accessi in memoria: N .
- Istruzioni di addizione: N .
- Tempo di calcolo asintotico: $\Theta(N)$.

1.5.2 Analisi nel Modello a Due Livelli

Assumiamo che la memoria secondaria memorizzi l'array A in modo consecutivo in blocchi di dimensione B e che $A[0]$ sia allineato con il confine di un blocco. Senza questa assunzione di allineamento, la scansione potrebbe richiedere al più un trasferimento di blocco aggiuntivo, senza alterare il limite asintotico:

1. Al primo accesso ad $A[0]$, si verifica un miss. Il sistema trasferisce il blocco contenente $A[0], \dots, A[B-1]$ nella memoria veloce con un singolo I/O.
2. I successivi $B-1$ accessi trovano i loro dati già nella memoria veloce, non richiedendo ulteriori I/O. Questo sfrutta la località spaziale.
3. Un nuovo trasferimento di I/O è necessario solo quando l'indice attraversa il confine di un blocco, ovvero, una volta ogni B elementi.

Pertanto, il costo totale di I/O di una scansione lineare è

$$\text{Scan}(N) = \left\lceil \frac{N}{B} \right\rceil = \Theta\left(\frac{N}{B}\right) \text{ operazioni di I/O.}$$

1.5.3 Accesso Lineare contro Accesso Sparso

- **Accesso sequenziale (alta località spaziale):** la lettura di N elementi richiede N/B operazioni di I/O. Poiché B varia tipicamente da 64 byte per una linea di cache a centinaia di kilobyte o megabyte per pagine e blocchi del disco, lo speedup rispetto al modello ingenuo può essere di un fattore approssimativamente pari a B .
- **Accesso casuale o passo $k > B$ (nessuna località spaziale):** ogni elemento richiesto potrebbe risiedere in un blocco diverso, causando N operazioni di I/O indipendenti. Il costo degrada a $\Theta(N)$ I/O, il che è asintoticamente più lento di un fattore B .

Capitolo 2

Accesso a Passi, B-Tree e Impatto della Paginazione Virtuale

Lezione 2

Accesso a passi, ricerca binaria contro B-Tree e paginazione virtuale

Data: 16 settembre 2026

Questo capitolo studia come il pattern di accesso di un algoritmo interagisca con la granularità di trasferimento di un sistema di memoria. Lo stesso lavoro aritmetico può avere costi molto differenti a seconda che gli accessi preservino la località spaziale, corrispondano alla dimensione del blocco, o superino la capacità della memoria veloce.

2.1 Accesso Non Sequenziale: L'Algoritmo

$A_{s,b}$

Sia S un array contenente n elementi. Il sistema di archiviazione trasferisce i dati in blocchi fisici di dimensione B (un blocco del disco o una linea di cache). Definiamo inoltre una dimensione del blocco logico b , dove $b \leq B$ e b divide B . I corrispondenti numeri di blocchi logici e fisici sono

$$N' = \frac{n}{b}, \quad N_{\text{phys}} = \frac{n}{B}.$$

L'algoritmo $A_{s,b}$ calcola la somma degli elementi visitando i blocchi logici con un pattern a passi (stride). Il blocco logico visitato al passo j è

$$\text{block}(j) = (js) \bmod N', \quad j = 0, 1, \dots, N' - 1.$$

2.1.1 Condizione di Coprimalità

La sequenza visita ogni blocco logico esattamente una volta se e solo se

$$\gcd\left(s, \frac{n}{b}\right) = 1.$$

In tal caso, la moltiplicazione per s è una permutazione dei residui modulo N' . Se il massimo comun divisore è $d > 1$, la sequenza si ripete dopo $N'/d = n/(bd)$ blocchi logici distinti e di conseguenza copre solo una frazione dell'array.

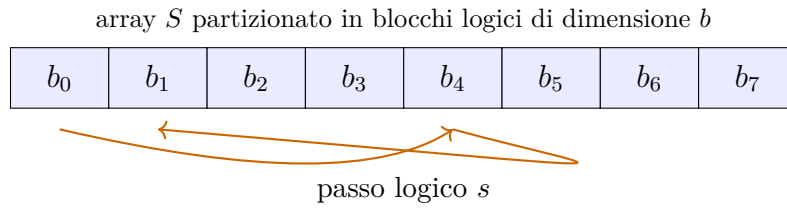


Figura 2.1: Un passo modulare visita i blocchi logici in un ordine non sequenziale.

La [Figura 2.1](#) illustra l'ordine logico degli accessi. Il costo fisico dipende da come questo ordine si mappa su blocchi di dimensione B , piuttosto che unicamente dal numero di operazioni aritmetiche.

2.1.2 Costo di I/O

Una prima utile approssimazione confronta la distanza fisica tra accessi logici successivi, sb , con la dimensione del blocco fisico B .

Tabella 2.1: Comportamento approssimato di I/O del pattern di accesso a passi.

Condizione	Località e comportamento	Costo di I/O
$sb < B$	Diversi accessi logici possono condividere un singolo blocco fisico; una parziale località spaziale è preservata.	$\Theta(n/B)$
$sb \geq B$	Gli accessi successivi tipicamente ricadono su blocchi fisici differenti. Se $M < n$, il working set non può rimanere interamente nella memoria veloce.	$\Theta(n/b)$
$b = 1$ e $s \geq B$	Ogni accesso logico è un singolo elemento e il passo distrugge il beneficio dei trasferimenti a blocchi.	$\Theta(n)$

I limiti nella [Tabella 2.1](#) descrivono il tipico regime del caso pessimo o dei capacity miss. Le costanti esatte dipendono dall'allineamento, dalla politica di rimpiazzo e dall'interazione tra la sequenza modulare e i blocchi fisici. Nell'ultimo caso, il costo è peggiore di un fattore pari a circa B rispetto ad una scansione lineare, che può sfruttare ogni elemento trasferito.

2.2 Limiti Algoritmici e Accelerazione Hardware

Fissiamo un tempo di esecuzione massimo ammissibile T_0 e assumiamo che l'hardware o il parallelismo accelerino il calcolo di un fattore K . La macchina accelerata può pertanto permettersi un tempo di esecuzione originale pari a

$$\frac{T(n)}{K} = T_0 \quad \Longleftrightarrow \quad T(n) = KT_0.$$

L'effetto sulla dimensione massima dell'input risolvibile dipende dal tasso di crescita di T .

Tabella 2.2: Effetto di un'accelerazione hardware di un fattore K sulla dimensione dell'input risolvibile.

Algoritmo	Complessità	Hardware di base	Hardware accelerato	Guadagno effettivo
A_1	cn	$n_1 = T_0/c$	$n'_1 = Kn_1$	Moltiplicativo, K
A_2	cn^2	$n_2 = \sqrt{T_0/c}$	$n'_2 = \sqrt{K} n_2$	Sublineare, $\sqrt{K}$
A_3	$c2^n$	$n_3 = \log_2(T_0/c)$	$n'_3 = n_3 + \log_2 K$	Additivo, $+\log_2 K$

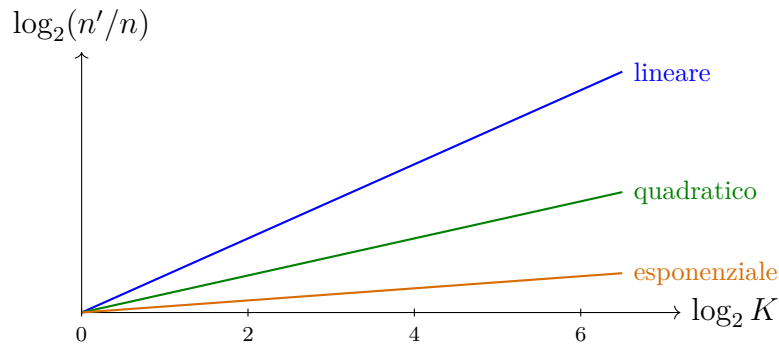


Figura 2.2: Beneficio relativo dell'accelerazione hardware per tassi di crescita rappresentativi.

La [Figura 2.2](#) mostra perché i miglioramenti hardware non rimuovono un algoritmo asintoticamente inefficiente. Per un algoritmo esponenziale, persino uno speedup di 1000 volte aumenta la dimensione dell'input risolvibile di soli $\log_2(1000) \approx 10$ elementi.

2.3 Ricerca in Insiemi Ordinati: Ricerca Binaria contro B-Tree

2.3.1 Ricerca Binaria in Memoria Secondaria

Per una sequenza ordinata di n chiavi memorizzate in blocchi di dimensione B , la ricerca binaria esegue $\Theta(\log_2 n)$ confronti sulla CPU. Il suo costo di I/O è maggiore di quanto il numero di confronti suggerisca: durante le prime iterazioni, le posizioni esaminate sono separate da $n/2$, $n/4$ e così via. Finché

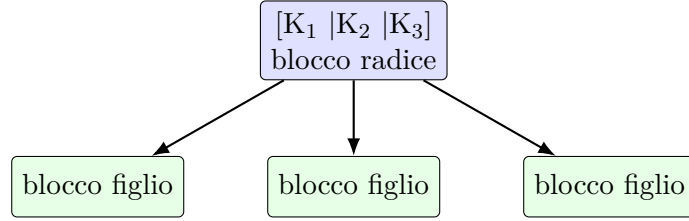
l'intervallo di ricerca non si restringe ad al più B elementi, ogni sondaggio può indirizzare un blocco fisico differente. Pertanto,

$$I/O_{\text{ricerca binaria}} \approx \log_2 n - \log_2 B = \log_2 \left(\frac{n}{B} \right).$$

L'approssimazione ignora l'allineamento e la possibilità che un blocco rimanga nella cache, ma cattura il comportamento dominante in memoria esterna.

2.3.2 B-Tree e B⁺-Tree

Un B-Tree sceglie la dimensione del nodo in modo che corrisponda ad un blocco fisico. Ogni nodo memorizza $\Theta(B)$ chiavi e puntatori ai figli, pertanto il suo fan-out è $\Theta(B)$. Un B⁺-Tree usa lo stesso principio e normalmente memorizza i record o i puntatori ai record nelle sue foglie, rendendolo particolarmente adatto per le query ad intervallo.



ogni nodo occupa approssimativamente un blocco fisico B

Figura 2.3: Un B-Tree orientato ai blocchi con alto fan-out.

La [Figura 2.3](#) mostra la decisione fondamentale di layout: un I/O carica un intero nodo e fornisce molte chiavi di instradamento e puntatori. L'altezza, e di conseguenza il numero di letture di blocchi per una point query, è approssimativamente

$$h = \log_B \left(\frac{n}{B} \right) = \frac{\log_2(n/B)}{\log_2 B}.$$

Più precisamente, i dettagli implementativi introducono arrotondamenti per eccesso e fattori di occupazione. Lo spazio dei nodi interni è piccolo in confronto a quello delle foglie poiché

$$\text{nodi interni} \leq \frac{n}{B^2} + \frac{n}{B^3} + \dots = \frac{n}{B(B-1)} \ll \frac{n}{B}.$$

2.3.3 Confronto Quantitativo

Ignorando gli arrotondamenti per eccesso, lo speedup di I/O di un B-Tree rispetto alla ricerca binaria è

$$\frac{I/O_{\text{ricerca binaria}}}{I/O_{\text{B-Tree}}} = \frac{\log_2(n/B)}{\log_2(n/B)/\log_2 B} = \log_2 B.$$

Tabella 2.3: Confronto illustrativo per un grande dataset ordinato.

Parametro	Valore
Dimensione del dataset	$n = 2^{40}$ elementi, approssimativamente 10^{12} record
Dimensione del blocco	$B = 2^{14} = 16\,384$ record
I/O della ricerca binaria	$40 - 14 = 26$ accessi ai blocchi
I/O del B-Tree	$(40 - 14)/14 \approx 1.85$, quindi circa 2 accessi ai blocchi

Per una latenza di archiviazione su scala dei millisecondi, la differenza tra approssimativamente 26 e 2 accessi ai blocchi cambia una point query da centinaia di millisecondi a soli pochi millisecondi, subordinatamente alla cache e ai dettagli hardware.

2.4 Algoritmi I/O-Aware e Paginazione Virtuale

Ci sono due modi complementari per gestire i trasferimenti tra memoria veloce e lenta:

- **Progettazione I/O-aware (esplicita):** L'algoritmo modella B e la capacità della memoria veloce M in modo diretto, e gestisce buffer, layout dei dati e trasferimenti in modo programmatico.
- **Memoria virtuale e paginazione (implicita):** Il sistema operativo e l'hardware muovono le pagine in modo trasparente utilizzando tabelle delle pagine e politiche di rimpiazzo come le approssimazioni LRU.

2.4.1 Paginazione con un Working Set Sovradimensionato

Assumiamo che la memoria fisica abbia capacità M e sia divisa in M/B frame. Se l'applicazione richiede

$$n = (1 + \varepsilon)M = M + \varepsilon M, \quad \varepsilon > 0,$$

allora almeno εM unità del working set devono risiedere nella memoria secondaria. Sia α la frazione di istruzioni che eseguono operazioni di memoria. Un tipico profilo di istruzione è:

- frazione $1 - \alpha$: operazioni aritmetiche o logiche, con costo di circa un ciclo di CPU;
- frazione α : operazioni LOAD o STORE;
- probabilità $1 - P(\varepsilon)$: i dati referenziati sono residenti in RAM, con tempo di accesso t_m ;
- probabilità $P(\varepsilon)$: l'accesso causa un page fault, con tempo di servizio t_{disco} .

Il tempo medio risultante per istruzione è

$$T_{\text{medio}} = (1 - \alpha) \cdot 1 + \alpha [(1 - P(\varepsilon))t_m + P(\varepsilon)t_{\text{disco}}].$$

Poiché $t_m \ll t_{\text{disco}}$, il termine del page fault può dominare:

$$T_{\text{medio}} \approx \alpha P(\varepsilon) t_{\text{disco}}.$$

Questa approssimazione è un avvertimento sulla sensibilità, non un'identità: quando $P(\varepsilon)$ è zero, i termini di base della CPU e della RAM hanno ancora rilevanza.

2.4.2 Esempio Numerico di Thrashing

Prendiamo $\alpha = 0.3$, una penalità per page fault di $t_{\text{disco}} = 10^5$ cicli di clock, e una probabilità di fault di soli $P(\varepsilon) = 10^{-3}$. Allora la penalità attesa è

$$T_{\text{penalità}} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30 \text{ cicli per istruzione.}$$

Un page fault ogni mille accessi in memoria può pertanto aggiungere in media circa 30 cicli ad ogni istruzione. Rispetto ad una baseline di un ciclo, questo può produrre un rallentamento di un ordine di grandezza, potenzialmente nel range di $10\times$ – $30\times$ a seconda della latenza RAM ordinaria e dell'effettivo comportamento del working set. Quando il tasso di fault aumenta ulteriormente, il processo entra in *thrashing*: il sistema spende la maggior parte del suo tempo a servire page fault invece di progredire con computazione utile.

2.5 Punti Chiave

1. Un passo modulare visita tutti i blocchi logici solo quando il passo e il numero di blocchi logici sono coprimi.
2. Layout e pattern di accesso block-aware possono cambiare la complessità di I/O di un fattore B , anche quando il lavoro della CPU rimane inalterato.
3. I B-Tree riducono la profondità di ricerca in memoria esterna facendo corrispondere il loro fan-out alla dimensione del blocco fisico.
4. L'accelerazione hardware migliora gli algoritmi lineari e polinomiali in modi differenti, ma fornisce solo un miglioramento additivo agli algoritmi esponenziali.
5. La memoria virtuale è conveniente, ma un working set sovradimensionato può trasformare una piccola probabilità di page fault in un costo prestazionale dominante.

Capitolo 3

Permutazione, Ordinamento e Multiway Merge Sort in Memoria Esterna

Lezione 3

Permutazione vs Ordinamento, Multiway Merge Sort, Snow Plow e Disk Striping

Data: 18 Settembre 2026

In questo capitolo viene esaminata la fondamentale relazione tra il problema della permutazione e quello dell'ordinamento nel modello di memoria a due livelli, confrontando i limiti del modello RAM con le gerarchie a blocchi. Viene introdotto e analizzato il Binary Merge Sort esterno evidenziandone i vincoli di sottoutilizzo, per poi sviluppare in dettaglio il *Multiway Merge Sort* (k -Way Merge Sort) che raggiunge il lower bound asintotico di Aggarwal e Vitter. Vengono infine approfondite due tecniche cardine dell'Algorithm Engineering su memoria secondaria: l'ottimizzazione dei run iniziali mediante *Snow Plow (Replacement Selection)* e la scalabilità parallela su più dischi mediante *Disk Striping*.

3.1 Il Problema della Permutazione vs. Ordinamento

3.1.1 Formulazione del Problema

Siano dati due vettori di dimensione n :

- Un array di dati $S[1 \dots n]$.
- Un vettore di permutazione $\pi[1 \dots n]$, contenente una permutazione biiettiva degli indici $\{1, 2, \dots, n\}$.

L'obiettivo consiste nel riordinare gli elementi di S in un nuovo array S' in modo tale che ogni elemento originario $S[i]$ sia ricollocato nella posizione target indicata da $\pi[i]$, ovvero:

$$S'[\pi[i]] = S[i], \quad \forall i \in \{1, \dots, n\}. \quad (3.1)$$

Esempio 3.1.1 (Permutazione di indici). Consideriamo un array di 4 elementi e il relativo vettore di permutazione:

$$\begin{array}{l} \text{Indici:} \quad 1 \quad 2 \quad 3 \quad 4 \\ S = \quad [A, \quad B, \quad C, \quad D] \\ \pi = \quad [3, \quad 1, \quad 2, \quad 4] \end{array}$$

Applicando la definizione elemento per elemento:

$$\begin{aligned} S'[\pi[1]] &= S'[3] = A, \\ S'[\pi[2]] &= S'[1] = B, \\ S'[\pi[3]] &= S'[2] = C, \\ S'[\pi[4]] &= S'[4] = D. \end{aligned}$$

L'array permutato finale risulta pertanto:

$$S' = [B, \quad C, \quad A, \quad D].$$

In [Figura 3.1](#) viene illustrata visivamente la mappatura degli elementi dall'array originale al vettore permutato.

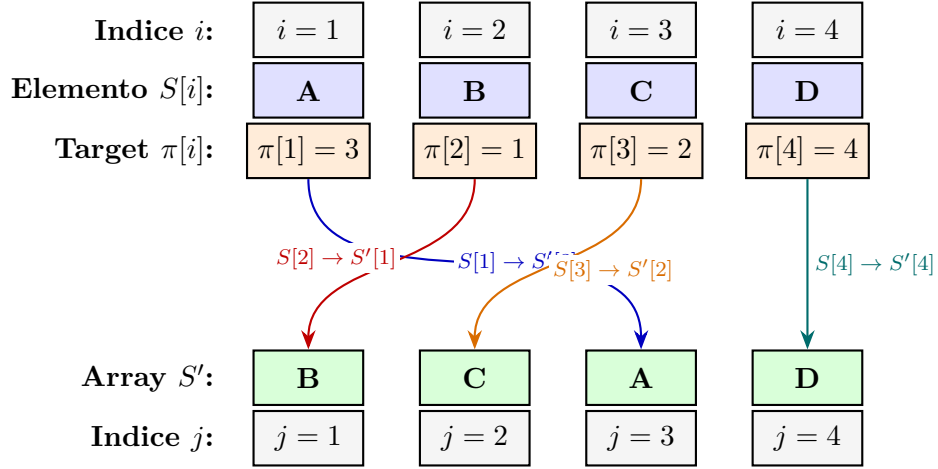


Figura 3.1: Mappatura della permutazione: ogni elemento $S[i]$ viene ricollocato nella posizione target specificata da $\pi[i]$ nell'array S' .

3.1.2 Implementazione Ingenua su Modello RAM

Nel modello RAM classico, in cui ogni accesso in lettura o scrittura a qualsiasi locazione di memoria ha costo unitario $\mathcal{O}(1)$, l'algoritmo si implementa tramite una singola scansione lineare:

Algorithm 1: Permutazione Diretta su RAM

Dati: Array $S[1 \dots n]$, vettore di permutazione $\pi[1 \dots n]$

Risultato: Array S permutato in loco

1 **for** $i \leftarrow 1$ **to** n **do**

2 $S'[\pi[i]] \leftarrow S[i]$

3 $S \leftarrow \text{copy}(S')$

La complessità temporale nel modello RAM è strettamente lineare:

$$T_{\text{RAM}}(n) = \Theta(n).$$

In memoria centrale, pertanto, permutare è asintoticamente più efficiente rispetto all'ordinamento basato su confronti, il quale richiede notoriamente un lower bound di $\Omega(n \log n)$ operazioni elementari.

3.1.3 Principio di Simulazione (RAM contro Due Livelli)

Teorema 3.1.1 (Teorema di Simulazione). *Qualsiasi algoritmo sequenziale formulato per il modello RAM con complessità temporale $T(n)$ richiede al più*

$\mathcal{O}(T(n))$ trasferimenti di blocchi (I/O) nel modello di memoria a due livelli con parametri (M, B) .

Dimostrazione (Idea). Nel caso peggiore, ogni istruzione elementare RAM che effettua un accesso in memoria (lettura o scrittura) indirizza una locazione appartenente a un blocco non presente in memoria veloce M . Ciascun accesso scatena pertanto un trasferimento di blocco dal disco, producendo esattamente 1 operazione di I/O. Moltiplicando il numero complessivo di passi $T(n)$ per il costo massimo di 1 I/O per passo, si ottiene il bound superiore immediato di $\mathcal{O}(T(n))$ I/O. $\square$

Sebbene il principio di simulazione garantisca che un algoritmo RAM possa sempre essere eseguito in memoria esterna, il bound $\mathcal{O}(T(n))$ è spesso inefficiente e lontano dall'ottimalità, poiché ignora completamente il fattore di aggregazione B offerto dai trasferimenti a blocchi.

3.1.4 La Relazione tra Ordinamento e Permutazione

Nel modello a due livelli la gerarchia di complessità si ribalta, come sintetizzato nel prospetto comparativo in [Tabella 3.1](#):

Problema	Modello RAM	Memoria a Due Livelli (I/O)
Permutazione Perm(n)	$\Theta(n)$	$\Theta\left(\min\left\{n, \text{Sort}(n, M, B)\right\}\right)$
Ordinamento Sort(n)	$\Theta(n \log_2 n)$	$\Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{B}\right)$

Tabella 3.1: Confronto di complessità asintotica tra RAM e Memoria Esterna.

Per comprendere questa equivalenza nei sistemi a blocchi, consideriamo le due riduzioni:

1. **Ordinamento tramite Permutazione:** Dato un array S , determinare la sequenza ordinata richiede dapprima di individuare la permutazione ordinante π_S (calcolabile tramite ordinamento delle chiavi) e successivamente di applicare π_S all'array S .
2. **Permutazione tramite Ordinamento (Algoritmo a Riduzione di Coppie):** La permutazione di un array S può essere trasformata in un problema di ordinamento di record secondo il seguente procedimento a tre stadi:

- **Passo 1 (Scansione e Creazione Coppie):** Si effettuano scansioni parallele e sequenziali di S e π per costruire una collezione di n record composti dalla coppia $\langle S[i], \pi[i] \rangle$. Poiché si tratta di una scansione sequenziale lineare, il costo è:

$$I/O_{\text{Passo 1}} = \mathcal{O}\left(\frac{n}{B}\right).$$

- **Passo 2 (Ordinamento per Destinazione):** Si ordinano le n coppie generate assumendo come chiave primaria la seconda componente $\pi[i]$. Il costo coincide esattamente con l'ordinamento in memoria esterna:

$$I/O_{\text{Passo 2}} = \text{Sort}(n, M, B).$$

- **Passo 3 (Scansione ed Estrazione):** Avendo ordinato le coppie per indice di destinazione crescente, è sufficiente una scansione lineare per estrarre la prima componente $S[i]$, riversandola nel vettore finale S' . Costo:

$$I/O_{\text{Passo 3}} = \mathcal{O}\left(\frac{n}{B}\right).$$

Il costo I/O aggregato di questa strategia è:

$$I/O_{\text{Sort-based}} = \mathcal{O}\left(\frac{n}{B}\right) + \text{Sort}(n, M, B) + \mathcal{O}\left(\frac{n}{B}\right) = \Theta(\text{Sort}(n, M, B)). \quad (3.2)$$

Esempio 3.1.2 (Tracciamento della Riduzione a Coppie con Array Reali). Per visualizzare concretamente la riduzione, consideriamo i dati dell'[Esempio 3.1.1](#):

$$S = [A, B, C, D], \quad \pi = [3, 1, 2, 4].$$

L'algoritmo opera attraverso tre stati successivi dell'array di record:

- **Passo 1 (Accoppiamento):** Si accoppia sequenzialmente ciascun elemento con la propria coordinata target:

$$P = [\langle A, 3 \rangle, \langle B, 1 \rangle, \langle C, 2 \rangle, \langle D, 4 \rangle].$$

- **Passo 2 (Sort esterno su $\pi[i]$):** L'array viene ordinato assumendo come chiave di comparazione il secondo valore della tupla (la destinazione finale):

$$P_{\text{ordinato}} = [\langle B, 1 \rangle, \langle C, 2 \rangle, \langle A, 3 \rangle, \langle D, 4 \rangle].$$

- **Passo 3 (Estrazione sequenziale):** Proiettando la prima coordinata di ciascun elemento ordinato, si ricompone il vettore finale:

$$S' = [B, C, A, D],$$

che realizza esattamente la permutazione originaria poiché $S'[1] = B$, $S'[2] = C$, $S'[3] = A$, $S'[4] = D$.

La pipeline di riduzione delle coppie è illustrata graficamente in [Figura 3.2](#).

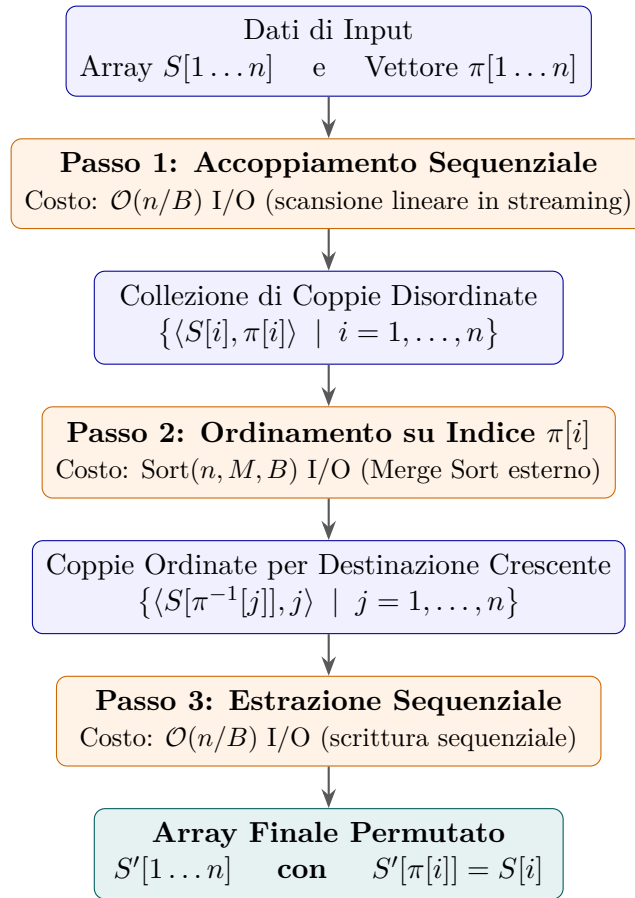


Figura 3.2: Flusso operativo dell'algoritmo di permutazione basato sulla riduzione a coppie.

3.2 Complessità di Permutazione in Memoria Esterna

Il lower bound fondamentale per il problema della permutazione in memoria a due livelli è stato stabilito da Aggarwal e Vitter [2]:

$$\text{Perm}(n, M, B) = \Theta \left(\min \left\{ n, \frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right\} \right). \quad (3.3)$$

Definendo $L = \log_{M/B}(n/B)$ come il numero di passate richieste dalla fusione multi-via nell'ordinamento:

$$\text{Sort}(n, M, B) = \Theta \left(\frac{n}{B} \cdot L \right). \quad (3.4)$$

3.2.1 Analisi dei Casi Limite

Il confronto tra l'algoritmo ingenuo diretto e la strategia basata sull'ordinamento delle coppie dipende dal valore relativo tra la dimensione del blocco B e il numero di livelli L :

- **Caso 1** ($n \geq \frac{n}{B} \cdot L \iff B \geq L$): La dimensione del blocco B è maggiore o uguale alla profondità dell'albero di fusione. In questa situazione, ordinare le coppie richiede meno I/O rispetto all'accesso casuale elemento per elemento:

$$\text{Perm}(n, M, B) = \Theta(\text{Sort}(n, M, B)).$$

- **Caso 2** ($n < \frac{n}{B} \cdot L \iff B < L$): Se la dimensione del blocco fosse patologicamente piccola rispetto al volume complessivo dei dati ($B < L$), eseguire n accessi diretti (al più n I/O) risulterebbe asintoticamente inferiore al costo dell'ordinamento:

$$\text{Perm}(n, M, B) = \Theta(n).$$

3.2.2 Verifica Numerica su Parametri Architettureali Reali

Per comprendere quale dei due regimi asintotici si manifesti nei sistemi reali, consideriamo parametri architetturali realistici ed estremi:

- Memoria interna veloce: $M = 16 \text{ GB} = 2^4 \cdot 2^{30} \text{ elementi} = 2^{34} \text{ elementi}$.

- Dimensione del blocco disco: $B = 32 \text{ KB} = 2^5 \cdot 2^{10} \text{ elementi} = 2^{15}$ elementi.
- Dimensione del dataset di input: $n = 2^{60}$ elementi (scala Exabyte).

Calcoliamo il numero di passate L :

$$L = \log_{M/B} \left(\frac{n}{M} \right) = \frac{\log_2(n/M)}{\log_2(M/B)}.$$

Sostituendo i valori esponenziali:

$$\begin{aligned} \frac{n}{M} &= \frac{2^{60}}{2^{34}} = 2^{26} \implies \log_2 \left(\frac{n}{M} \right) = 26, \\ \frac{M}{B} &= \frac{2^{34}}{2^{15}} = 2^{19} \implies \log_2 \left(\frac{M}{B} \right) = 19. \end{aligned}$$

Otteniamo:

$$L = \frac{26}{19} \approx 1,36 < 2.$$

Poiché $B = 2^{15} = 32\,768$ e $L \leq 2$, la condizione $B \geq L$ è verificata con un margine di oltre quattro ordini di grandezza ($32\,768 \gg 2$). Di conseguenza, in qualsiasi sistema reale la permutazione è strettamente governata dal costo di ordinamento:

$$\text{Perm}(n) = \Theta(\text{Sort}(n)).$$

3.3 Binary Merge Sort in Memoria Esterna

L'algoritmo ricorsivo classico di Merge Sort si estende alla memoria esterna suddividendo l'array in due metà, ordinando ricorsivamente ciascuna metà e fondendo i due sottoarray ordinati (*runs*):

Algorithm 2: Binary Merge Sort

Dati: Array S , estremi i, j

Risultato: Sottoarray $S[i \dots j]$ ordinato

```

1 if  $i < j$  then
2    $m \leftarrow \lfloor (i + j)/2 \rfloor$ 
3    $\text{MS}(S, i, m)$ 
4    $\text{MS}(S, m + 1, j)$ 
5    $\text{MERGE}(S, i, m, j)$ 
```

3.3.1 Architettura dei Buffer nella Procedura di Fusione

Dati due segmenti ordinati (*runs*) consecutivi di lunghezza l , la procedura di fusione genera un unico run ordinato di lunghezza $2l$ su un array temporaneo S' .

- **Nel Modello RAM:** Si effettuano al più $2l$ confronti e $2l$ trasferimenti in memoria, determinando un costo computazionale di $\Theta(l)$.
- **Nel Modello a Due Livelli:** Per eseguire la fusione minimizzando gli I/O, la memoria interna M viene partizionata in 3 buffer dedicati, ciascuno di dimensione esattamente pari a un blocco B :
 1. **Input Buffer 1:** ospita un blocco (dimensione B) del primo run ordinato.
 2. **Input Buffer 2:** ospita un blocco (dimensione B) del secondo run ordinato.
 3. **Output Buffer:** raccoglie gli elementi fusi in ordine crescente fino al riempimento di un blocco B .

In Figura 3.3 è rappresentata la gestione della memoria interna e dei flussi da/verso il disco durante la fusione binaria.

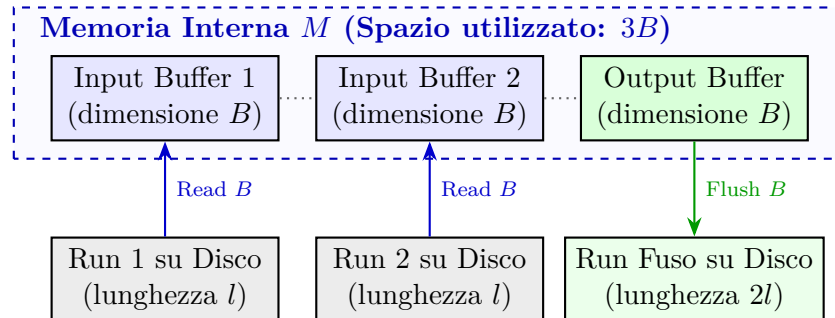


Figura 3.3: Architettura dei buffer di memoria interna durante la procedura di fusione binaria.

Il funzionamento del protocollo di trasferimento è il seguente:

1. Gli elementi in testa ai due buffer di input vengono confrontati in CPU; l'elemento minore viene spostato nell'output buffer.

2. Quando uno dei buffer di input si svuota, viene emessa una lettura da disco per caricare il blocco successivo del rispettivo run ($\frac{l}{B}$ I/O per ciascun run di input).
3. Non appena l'output buffer si riempie con B elementi, viene scaricato (*flushed*) su disco con una singola scrittura ($\frac{2l}{B}$ I/O complessivi di scrittura).

Il costo aggregato della fusione di due run di lunghezza l è:

$$\text{I/O}_{\text{Merge}}(l) = \frac{l}{B} + \frac{l}{B} + \frac{2l}{B} = \frac{4l}{B} = \Theta\left(\frac{l}{B}\right). \quad (3.5)$$

3.3.2 Complessità dell'Albero di Ricorsione

La ricorsione si arresta non appena la dimensione del sottoarray scende al di sotto della capacità della memoria interna M . A questo livello base, ciascun sottoarray di dimensione M viene letto interamente in memoria, ordinato localmente tramite un algoritmo interno efficiente (es. QuickSort o HeapSort) con zero I/O aggiuntivi, e riscritto su disco con $\mathcal{O}(M/B)$ I/O.

Si generano così inizialmente:

$$N_{\text{run}} = \frac{n}{M} \text{ run ordinati di dimensione } M.$$

L'albero di fusione binaria presenta un'altezza complessiva pari a:

$$h = \log_2\left(\frac{n}{M}\right) \text{ livelli.} \quad (3.6)$$

In Figura 3.4 è illustrata la struttura dell'albero di fusione per il Binary Merge Sort esterno.

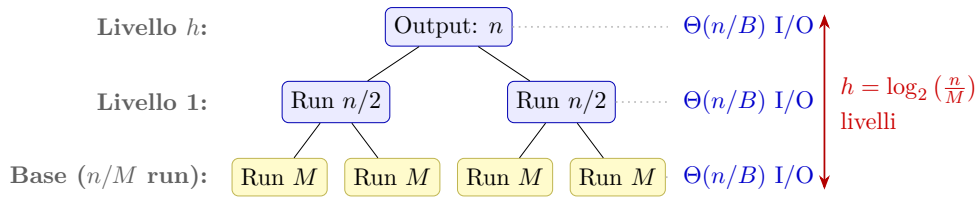


Figura 3.4: Albero di ricorsione del Binary Merge Sort esterno con indicazione dei costi I/O per livello.

Ad ogni livello dell'albero di fusione, tutti gli n elementi dell'array vengono scanditi esattamente una volta, richiedendo $\Theta(n/B)$ operazioni di I/O per livello. Moltiplicando per l'altezza dell'albero si ottiene la complessità totale:

$$\text{I/O}_{\text{BinaryMergeSort}} = \mathcal{O}\left(\frac{n}{B} \log_2\left(\frac{n}{M}\right)\right). \quad (3.7)$$

3.3.3 Criticità e Limiti di Efficienza

Nonostante l'algoritmo migliori drasticamente rispetto a un approccio ingenuo, esso presenta un grave collo di bottiglia strutturale:

- **Sottoutilizzo drastico della memoria interna:** Durante la fusione binaria vengono occupati contemporaneamente soltanto 3 blocchi ($3B$ elementi) all'interno dello spazio disponibile M . Tutta la restante porzione di memoria interna ($M - 3B$) rimane completamente inutilizzata.
- **Base del logaritmo sub-ottimale:** La base del logaritmo è rigidamente ancorata a 2, mentre la memoria M permetterebbe di mantenere aperti simultaneamente fino a:

$$k \approx \frac{M}{B} - 1 \text{ buffer di input.}$$

- **Perdita del fattore di convergenza $\log_2(M/B)$:** La mancata estensione a una fusione a k vie (*Multiway Merge Sort*) comporta un numero di passate significativamente superiore, impedendo di raggiungere il lower bound ottimale:

$$\mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right).$$

In Figura 3.5 viene evidenziata graficamente la disparità tra l'allocazione binaria e la piena saturazione della memoria veloce.

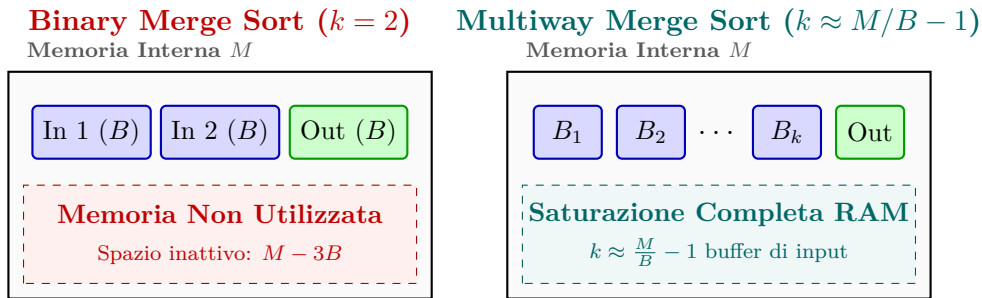


Figura 3.5: Confronto dell'efficienza nell'allocazione della memoria interna: grave sottoutilizzo nel Binary Merge Sort rispetto alla saturazione ottimale nel Multiway Merge Sort.

Il superamento di questa limitazione strutturale conduce in modo naturale all'architettura dell'ordinamento multi-via, analizzata in dettaglio nelle sezioni seguenti.

3.4 Equivalenza Asintotica del Costo di Ordinamento Esterno

Nella trattazione della complessità computazionale in memoria secondaria, il costo I/O dell'ordinamento esterno viene espresso nella letteratura specializzata [2] attraverso due notazioni apparentemente distinte:

$$\Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right)\right) \quad \text{oppure} \quad \mathcal{O}\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right). \quad (3.8)$$

Queste due formulazioni sono asintoticamente perfettamente equivalenti per qualsiasi istanza $n \geq M$.

Per dimostrare formalmente l'equivalenza, applichiamo le proprietà fondamentali dei logaritmi espandendo l'argomento del termine $\log_{M/B}(n/B)$:

$$\begin{aligned} \log_{M/B}\left(\frac{n}{B}\right) &= \log_{M/B}\left(\frac{n}{M} \cdot \frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + \log_{M/B}\left(\frac{M}{B}\right) \\ &= \log_{M/B}\left(\frac{n}{M}\right) + 1. \end{aligned} \quad (3.9)$$

Moltiplicando l'identità per il fattore comune $\frac{n}{B}$, otteniamo:

$$\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right) = \frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right) + \frac{n}{B}. \quad (3.10)$$

Dal punto di vista algoritmico, il termine additivo lineare $\frac{n}{B}$ corrisponde esattamente alla passata iniziale (*Passo 0*) necessaria alla formazione dei *run* ordinati di partenza tramite caricamento e ordinamento in RAM.

Allorché $n \gg M$, si ha $\log_{M/B}(n/M) \geq 1$, per cui il termine additivo $+1$ viene pienamente assorbito dalla costante nascosta nella notazione asintotica Θ :

$$\Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{B}\right)\right) = \Theta\left(\frac{n}{B} \left(\log_{M/B}\left(\frac{n}{M}\right) + 1\right)\right) = \Theta\left(\frac{n}{B} \log_{M/B}\left(\frac{n}{M}\right)\right). \quad (3.11)$$

Ne consegue che è possibile utilizzare indifferentemente $\frac{n}{M}$ oppure $\frac{n}{B}$ all'interno dell'argomento logaritmico senza alterare la rigorosità dell'analisi.

3.5 Multiway Merge Sort (*k*-Way Merge Sort)

Nel *Binary Merge Sort*, descritto nella [sezione 3.3](#), si allocano simultaneamente soltanto 3 blocchi di memoria interna, lasciando inutilizzato lo spazio residuo

$M - 3B$. Il **Multiway Merge Sort** (o ordinamento per fusione a k vie) supera radicalmente questa inefficienza sfruttando l'intera capacità M per fondere simultaneamente k sequenze ordinate in un'unica passata (*fan-in* pari a k).

3.5.1 Configurazione e Architettura della Memoria Interna

La memoria primaria M viene partizionata in unità di trasferimento di dimensione B :

- $k = \frac{M}{B} - 1$ **buffer di input**, ciascuno di capacità esattamente pari a un blocco B , associati stabilmente a ciascuno dei k run attivi su memoria secondaria.
- 1 **buffer di output**, anch'esso di capacità B , deputato ad accumulare progressivamente gli elementi risultanti in ordine crescente prima del trasferimento fisico su disco.
- Una struttura dati ausiliaria residente interamente in memoria interna: un **Min-Heap** (o coda di priorità) contenente k nodi. Ciascun nodo memorizza la chiave corrente minima di uno dei k blocchi attivi insieme all'identificatore del buffer di provenienza.

In Figura 3.6 viene illustrato lo schema architetturico dei buffer e il flusso dei dati coordinato dal Min-Heap.

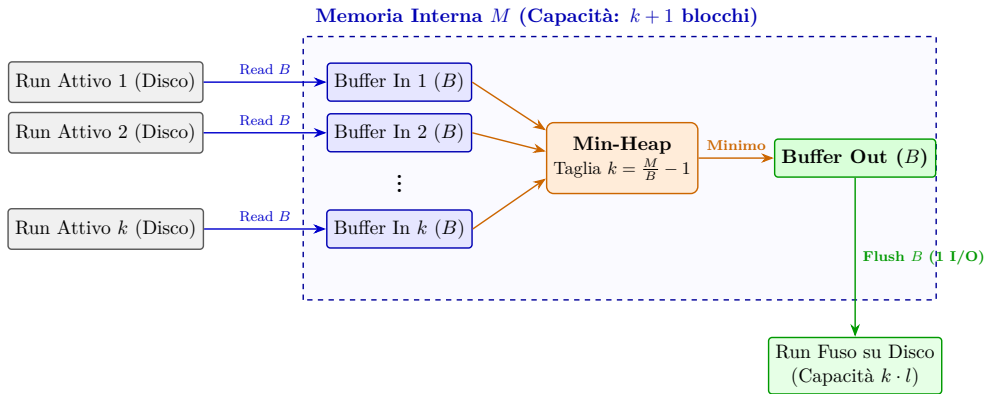


Figura 3.6: Architettura della memoria interna durante la fusione multi-via (k -Way Merge Sort) con Min-Heap di coordinamento.

Il Min-Heap consente di estrarre l'elemento minimo globale tra i k flussi con costo computazionale in memoria interna pari a $\mathcal{O}(\log_2 k)$ confronti. Quando

il buffer di input da cui proviene il minimo estratto si esaurisce, viene emessa una singola operazione di lettura I/O per caricare il successivo blocco da B elementi del corrispondente run. Analogamente, ogni qualvolta il buffer di output raccoglie B elementi ordinati, viene eseguito un *flush* fisico su disco tramite una singola scrittura I/O.

3.5.2 Dinamica dell'Algoritmo ed Evoluzione dei Livelli

L'algoritmo si articola in due macro-fasi strutturate:

1. **Fase 0 (Formazione dei Run Iniziali):** L'array non ordinato di n elementi residenti su disco viene scandito a porzioni contigue di dimensione massima gestibile in memoria veloce, pari a M . Ciascun blocco di M elementi viene caricato in RAM, ordinato localmente con un algoritmo in-memory efficiente (es. QuickSort o HeapSort) e riscritto ordinato su disco:

- Numero di run iniziali generati:

$$N_0 = \frac{n}{M}. \quad (3.12)$$

- Costo I/O della Fase 0: lettura e scrittura sequenziale di tutti gli n elementi, pari a:

$$\text{I/O}_{\text{RunFormation}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \quad (3.13)$$

2. **Fasi Successive di Fusione (k -Way Merge):** A ogni livello dell'albero di fusione, i run disponibili vengono raggruppati a gruppi di k . Ciascun gruppo di k run, ciascuno di lunghezza corrente l , viene fuso simultaneamente producendo un nuovo run ordinato di estensione $k \cdot l$:

- Costo di I/O per fondere un singolo gruppo: $\mathcal{O}\left(\frac{k \cdot l}{B}\right)$.
- Costo I/O complessivo per livello: poiché ogni elemento dell'array appartiene a esattamente un gruppo a ciascun livello, tutti gli n elementi vengono letti e riscritti esattamente una volta per ogni passata, con un consumo di:

$$\text{I/O}_{\text{Livello}} = 2 \cdot \frac{n}{B} = \mathcal{O}\left(\frac{n}{B}\right). \quad (3.14)$$

All' i -esimo livello dell'albero di fusione, ciascun run possiede una dimensione pari a $k^i \cdot M$. Il procedimento ricorsivo termina non appena rimane un unico run complessivo di dimensione n :

$$k^i \cdot M = n \implies k^i = \frac{n}{M} \implies i = \log_k \left(\frac{n}{M} \right) \text{ livelli.} \quad (3.15)$$

Sostituendo il massimo grado di fusione ammissibile $k \approx \frac{M}{B} - 1 = \Theta \left(\frac{M}{B} \right)$, otteniamo:

$$\text{Numero di Fasi di Merge} = \mathcal{O} \left(\log_{M/B} \left(\frac{n}{M} \right) \right). \quad (3.16)$$

In [Figura 3.7](#) viene illustrata la gerarchia dei livelli di fusione a k vie, evidenziando il fan-in espanso e i costi di I/O sostenuti a ciascun livello dell'albero.

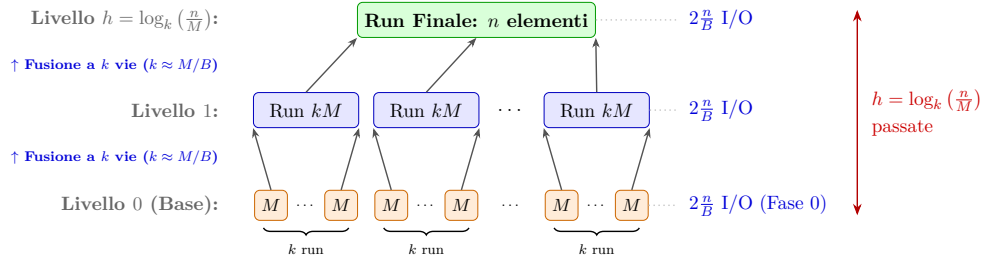


Figura 3.7: Evoluzione dei livelli e albero di fusione a k vie nel Multiway Merge Sort.

La complessità I/O complessiva dell'algoritmo risulta pertanto:

$$\begin{aligned} \text{I/O}_{\text{Totale}} &= \underbrace{\mathcal{O} \left(\frac{n}{B} \right)}_{\text{Run Formation}} + \underbrace{\mathcal{O} \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{M} \right) \right)}_{\text{Fasi di Fusione}} \\ &= \mathcal{O} \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right) \text{ I/O.} \end{aligned} \quad (3.17)$$

Questo risultato eguaglia asintoticamente il lower bound teorico per l'ordinamento in memoria gerarchica derivato da Aggarwal e Vitter [2], dimostrando l'ottimalità del Multiway Merge Sort.

3.6 Ottimizzazione dei Run Iniziali: Lo Spazaneve (*Snow Plow*)

Sebbene il Multiway Merge Sort standard generi run iniziali di dimensione limitata alla capacità della memoria interna M , l'altezza dell'albero di fusione

$\lceil \log_{M/B}(n/M) \rceil$ dipende in misura cruciale dal numero iniziale di sequenze prodotte. Produrre run di lunghezza maggiore di M consente di ridurre la profondità della ricorsione, eliminando intere passate di I/O su scala multi-terabyte.

Una tecnica classica ed elegante concepita a questo scopo è la **Replacement Selection**, nota storicamente con la suggestiva metafora dello **Spazzaneve** (*Snow Plow*) [1].

3.6.1 Meccanismo Operativo della Replacement Selection

Invece di riempire staticamente la memoria M , ordinarla e scaricarla per intero, l'algoritmo mantiene costantemente saturata la memoria primaria M partizionandola dinamicamente in due insiemi logici:

- **Parte Attiva (Min-Heap corrente):** elementi che risultano maggiori o uguali all'ultimo elemento emesso verso il run in fase di scrittura. Tali elementi sono candidati legittimi ad arricchire il run corrente.
- **Parte Congelata o Disordinata (U):** elementi letti dal flusso di ingresso che risultano strettamente minori dell'ultimo elemento emesso. Essi non possono essere accodati al run corrente senza violarne l'ordinamento e vengono dunque “congelati”, rimanendo parcheggiati in RAM in attesa della formazione del run successivo.

In [Figura 3.8](#) viene rappresentata la partizione dinamica dello spazio in memoria primaria durante l'avanzamento dello spazzaneve.

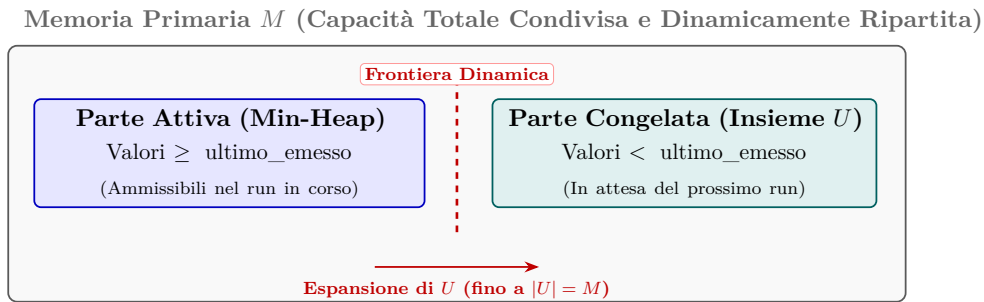


Figura 3.8: Ripartizione dinamica della memoria M durante la Replacement Selection (Spazzaneve).

Il ciclo operativo procede elemento per elemento:

1. Si estrae l'elemento minimo dalla parte attiva tramite la radice del Min-Heap, lo si invia al buffer di output e si registra il suo valore come `ultimo_emesso`.
2. Si legge un nuovo elemento x dal flusso di input in ingresso:
 - Se $x \geq \text{ultimo_emesso}$, x viene inserito nella parte attiva dell'heap.
 - Se $x < \text{ultimo_emesso}$, x viene inserito nella partizione dei congelati U .
3. Il run corrente termina quando la parte attiva si svuota completamente, ossia quando tutti gli M slot di memoria risultano occupati da elementi congelati ($|U| = M$). A questo punto, l'insieme U viene riconvertito istantaneamente nel nuovo Min-Heap attivo, si apre un nuovo run su disco e il processo riparte.

3.6.2 Esempio Passo-Passo dell'Algoritmo

Consideriamo una memoria rapida di capacità $M = 3$ elementi e una sequenza di input non ordinata:

$$S = [1, 7, 5, 3, 2, 9, 0, 4, \dots].$$

In [Tabella 3.2](#) viene tracciata dettagliatamente l'evoluzione dello stato interno della memoria e dei run emessi.

Tabella 3.2: Traccia dell'algoritmo Replacement Selection con capacità $M = 3$.

Passo	Attivi	Congelati (U)	Azione Eseguita	Emesso	Run
0	{1, 5, 7}	$\emptyset$	Caricamento di $M = 3$ elementi	—	[]
1	{5, 7}	$\emptyset$	Emette 1; legge 3. Poiché $3 \geq 1$, resta attivo	1	[1]
2	{5, 7}	$\emptyset$	Emette 3; legge 2. Poiché $2 < 3$, 2 va in U	3	[1, 3]
3	{7, 9}	{2}	Emette 5; legge 9. Poiché $9 \geq 5$, resta attivo	5	[1, 3, 5]
4	{9}	{2, 0}	Emette 7; legge 0. Poiché $0 < 7$, 0 va in U	7	[1, 3, 5, 7]
5	$\emptyset$	{2, 0, 4}	Emette 9; legge 4. Poiché $4 < 9$, 4 va in U	9	[1, 3, 5, 7, 9]

Al passo 5, la memoria attiva si azzerà ed è interamente saturata dai congelati: $|U| = M = 3$. Il run corrente si chiude formalmente con una lunghezza pari a **5 elementi**, sensibilmente superiore alla capacità nominale della RAM ($M = 3$). L'insieme $\{0, 2, 4\}$ forma immediatamente l'heap di avvio del secondo run ordinato.

Nota sui Sistemi Reali. Nei sistemi operativi e DBMS reali, il caricamento e l'emissione non avvengono elemento per elemento ma a blocchi fisici di dimensione B , integrando opportuni buffer di prefetching in lettura e di accumulo in scrittura (*double buffering*), preservando intatta l'efficienza asintotica di I/O.

3.6.3 Teorema della Lunghezza Media dei Run

Teorema 3.6.1 (Lunghezza Media del Run nello Snow Plow). *Assumendo che le chiavi in ingresso siano variabili aleatorie indipendenti e identicamente distribuite secondo una distribuzione continua uniforme, la lunghezza media dei run generati dall'algoritmo Snow Plow è asintoticamente pari a $2M$.*

Dimostrazione Euristica. Analizziamo il comportamento stocastico del sistema a regime:

1. All'avvio della formazione di un run, la memoria contiene M elementi ordinati.
2. Sia τ il numero di elementi addizionali letti dallo stream prima che il run corrente si concluda. Il numero complessivo di elementi appartenenti al run risulterà pari a $M + \tau$.
3. Il run si interrompe esattamente nel momento in cui la partizione dei congelati raggiunge la saturazione: $|U| = M$.
4. Per simmetria distribuzionale e indipendenza statistica, ogni nuovo elemento estratto ha probabilità pari a $P = \frac{1}{2}$ di posizionarsi a monte dell'ultimo valore emesso (venendo assegnato ai congelati U) e probabilità $P = \frac{1}{2}$ di risultare maggiore o uguale (entrando nella parte attiva del run corrente).
5. Il valore atteso del numero di elementi inviati a U dopo aver processato τ ingressi è dunque:

$$\mathbb{E}[|U|] = \tau \cdot P(\text{assegnato a } U) = \tau \cdot \frac{1}{2}. \quad (3.18)$$

6. Imponendo il vincolo di chiusura del run $\mathbb{E}[|U|] = M$, si ottiene:

$$\frac{\tau}{2} = M \implies \tau = 2M. \quad (3.19)$$

Pertanto, il numero medio atteso di elementi generati per ciascun run ammonta a circa $2M$. □

Raddoppiare la lunghezza media dei run ($\approx 2M$) dimezza il numero iniziale di run prodotti $\left(\frac{n}{2M}\right)$, riducendo concretamente il numero di passate globali sull'intero dataset.

3.7 Parallelismo su Più Dischi: *Disk Striping*

Nei sistemi di calcolo ad elevate prestazioni e nei moderni centri dati, lo storage secondario non è affidato a un unico supporto magnetico o a stato solido, ma a un'architettura parallela composta da D dischi fisici indipendenti.

3.7.1 Lower Bound di Ordinamento Parallelo

Quando un sistema dispone di D unità disco operanti in parallelo, ciascuna in grado di trasferire un blocco B per ciclo di I/O, il limite inferiore asintotico di Aggarwal e Vitter [2] per l'ordinamento scala con un fattore ideale D a denominatore:

$$\text{Sort}_D(n, M, B) = \Omega \left(\frac{n}{D \cdot B} \log_{M/B} \left(\frac{n}{M} \right) \right) \text{ I/O paralleli.} \quad (3.20)$$

Tuttavia, progettare un algoritmo che raggiunga tale limite inferiore gestendo D dischi indipendenti in modo asincrono risulta estremamente complesso: è necessario evitare il fenomeno del *bottlenecking* (o congestione dei dischi), in cui le letture si concentrano sbilanciatamente su una singola unità lasciando le restanti $D - 1$ inattive.

3.7.2 L'Architettura del Disk Striping

Per ottenere i vantaggi prestazionali del parallelismo evitando la complessità di una schedulazione asincrona disaccoppiata, si ricorre alla tecnica del **Disk Striping** (concettualmente coincidente con lo schema di archiviazione RAID 0):

- I D dischi fisici indipendenti $d_1, d_2, \dots, d_D$ vengono raggruppati logicamente in una singola macro-unità.
- Viene definito un **super-blocco logico** di dimensione aggregata:

$$B' = D \cdot B, \quad (3.21)$$

costituito dall'unione sincronizzata di un blocco fisico di taglia B allocato al medesimo indirizzo (o cilindro) su ciascuno dei D dischi.

- Qualsiasi operazione di I/O trasferisce sempre un intero super-blocco di ampiezza B' distribuendo il carico uniformemente e simultaneamente su tutti i D canali hardware in un singolo ciclo di clock I/O parallelo.

In [Figura 3.9](#) è rappresentata la mappatura del super-blocco B' sui dischi fisici sottostanti.

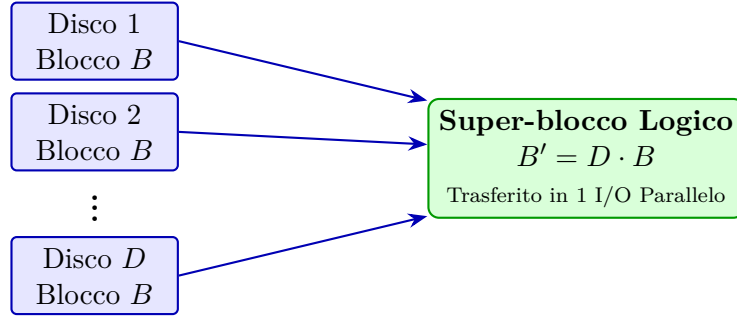


Figura 3.9: Architettura a Disk Striping: raggruppamento di D blocchi fisici in un super-blocco B' .

Applicando il Multiway Merge Sort assumendo la taglia del blocco pari a B' , il numero di super-blocchi da scansionare è $\frac{n}{B'} = \frac{n}{D \cdot B}$, mentre la capacità di fan-in di memoria diventa $\frac{M}{B'} = \frac{M}{D \cdot B}$. Il costo I/O parallelo complessivo assume la forma:

$$I/O_{\text{Striping}} = \mathcal{O} \left(\frac{n}{B'} \log_{M/B'} \left(\frac{n}{M} \right) \right) = \mathcal{O} \left(\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}} \left(\frac{n}{M} \right) \right). \quad (3.22)$$

3.7.3 Analisi del Rallentamento (*Slowdown*)

Sebbene il Disk Striping riduca il volume di trasferimenti fisici di un fattore D , esso restringe drasticamente il fan-in a disposizione dell'algoritmo da $\frac{M}{B}$ a $\frac{M}{D \cdot B}$, poiché la memoria deve allocare buffer di ampiezza multipla $B' = D \cdot B$.

Per quantificare la perdita di efficienza rispetto all'algoritmo ideale a dischi indipendenti, valutiamo il rapporto di rallentamento (*Slowdown*):

$$\text{Slowdown} = \frac{I/O_{\text{Striping}}}{I/O_{\text{Ideale}}} = \frac{\frac{n}{D \cdot B} \log_{\frac{M}{D \cdot B}} \left(\frac{n}{M} \right)}{\frac{n}{D \cdot B} \log_{\frac{M}{B}} \left(\frac{n}{M} \right)} = \frac{\log_{\frac{M}{D \cdot B}} \left(\frac{n}{M} \right)}{\log_{\frac{M}{B}} \left(\frac{n}{M} \right)}. \quad (3.23)$$

Esprimendo entrambi i logaritmi in base 2 mediante la formula di cambio base $\log_a x = \frac{\log_2 x}{\log_2 a}$:

$$\text{Slowdown} = \frac{\frac{\log_2(n/M)}{\log_2(M/(D \cdot B))}}{\frac{\log_2(n/M)}{\log_2(M/B)}} = \frac{\log_2(M/B)}{\log_2(M/(D \cdot B))} = \frac{\log_2(M/B)}{\log_2(M/B) - \log_2 D}. \quad (3.24)$$

Dividendo numeratore e denominatore per la quantità comune $\log_2(M/B)$:

$$\text{Slowdown} = \frac{1}{1 - \frac{\log_2 D}{\log_2(M/B)}}. \quad (3.25)$$

3.7.4 Discussione dei Regimi Asintotici e Limiti Fisici

L'efficienza del Disk Striping è interamente regolata dal rapporto adimensionale:

$$\rho = \frac{\log_2 D}{\log_2(M/B)}. \quad (3.26)$$

Si delineano due comportamenti estremi distinti:

1. **Regime di Memoria Abbondante** ($M \rightarrow \infty$ oppure $D \ll \frac{M}{B}$):
Quando la memoria interna è molto capiente rispetto al numero di dischi impiegati, la capacità di fan-in della RAM domina marcatamente:

$$\rho = \frac{\log_2 D}{\log_2(M/B)} \rightarrow 0 \implies \text{Slowdown} \rightarrow \frac{1}{1 - 0} = 1. \quad (3.27)$$

In questa circostanza il fattore di rallentamento è trascurabile: il Disk Striping consegue la medesima efficienza asintotica del sistema teorico a canali disaccoppiati, risultando una soluzione ottimale dal punto di vista ingegneristico per la sua estrema semplicità di implementazione.

2. **Saturazione della Memoria Primaria** ($\log_2 D \rightarrow \log_2(M/B)$):
Allorché il numero di dischi fisici D scala fino a raggiungere la cardinalità massima di blocchi allocabili in memoria ($D \approx \frac{M}{B}$), il rapporto ρ tende a 1. Di conseguenza, il denominatore di (3.25) tende a zero e la funzione diverge positivamente:

$$\lim_{\rho \rightarrow 1^-} \frac{1}{1 - \rho} = +\infty. \quad (3.28)$$

Interpretazione fisica: Se $D \geq \frac{M}{B}$, la memoria interna M non possiede spazio sufficiente nemmeno per ospitare un singolo super-blocco $B' = D \cdot B$ per ciascun flusso, determinando l'impossibilità di operare e il collasso strutturale dell'algoritmo di striping.

Capitolo 4

Limiti Inferiori per l'Ordinamento, Quicksort Dual-Pivot e Bounded Quicksort

Lezione 4

*Limiti Inferiori (RAM vs Modello a Due Livelli), Quicksort Dual-Pivot e
Bounded Quicksort*

Data: 25 Settembre 2026

In questo capitolo viene affrontata l'analisi teorica e ingegneristica dei limiti inferiori di complessità per il problema fondamentale dell'ordinamento basato su confronti, confrontando il modello RAM classico con il modello a due livelli di Aggarwal e Vitter. Successivamente, l'indagine si sposta sulle moderne evoluzioni architetturali dell'algoritmo *Quicksort*: viene analizzato il problema dei valori duplicati e l'algoritmo *Dual-Pivot Quicksort* introdotto da Yaroslavskiy, per poi esaminare formalmente la variante *Bounded Quicksort* (BQS), dimostrando come l'eliminazione della ricorsione in coda (*tail recursion elimination*) e il partizionamento controllato garantiscano uno spazio ausiliario rigorosamente limitato a $\Theta(\log_2 n)$ unito alla massima efficienza di cache tramite cutoff a *Insertion Sort*.

4.1 Modello RAM: Limiti Inferiori e Alberi di Decisione

Nel modello di calcolo RAM (*Random Access Machine*) standard, un algoritmo basato su confronti (*comparison-based algorithm*) non può accedere alla rappresentazione binaria interna delle chiavi per estrarne cifre o bit (come invece avviene negli algoritmi di tipo *Radix Sort*), ma può acquisire informazioni sull'ordine reciproco degli elementi esclusivamente attraverso confronti binari del tipo:

$$A < B, \quad A = B, \quad \text{oppure} \quad A > B. \quad (4.1)$$

Assumendo, senza perdita di generalità per l'analisi del limite inferiore, che tutte le chiavi in ingresso siano reciprocamente distinte, ogni confronto elementare ammette un esito strettamente dicotomico ($<$ oppure $>$).

4.1.1 Il Modello dell'Albero di Decisione

La classe degli algoritmi deterministici basati su confronti viene formalmente modellata mediante la struttura astratta dell'**Albero di Decisione** (*Decision Tree*):

- **Nodi interni:** Ciascun nodo interno rappresenta un singolo confronto elementare tra una coppia di elementi o posizioni dell'input, denotato convenzionalmente con $[A : B]$. Poiché le chiavi sono distinte, dal nodo si dipartono esattamente due archi uscenti, corrispondenti agli esiti possibili: l'arco sinistro associato all'evento $A < B$ e l'arco destro associato all'evento $A > B$. L'albero risulta pertanto un albero binario proprio.
- **Cammino radice-foglia:** Per ogni specifica istanza di input di dimensione n , l'algoritmo deterministico segue un unico cammino orientato discendente dalla radice verso una foglia. Ciascun nodo visitato lungo il cammino corrisponde a un confronto eseguito; la lunghezza del cammino (numero di archi attraversati) riflette il numero di confronti effettuati per quella specifica configurazione. La lunghezza del cammino più lungo dalla radice a una foglia coincide con l'**altezza** t dell'albero di decisione e quantifica il tempo di esecuzione nel caso peggiore $T(n)$.
- **Foglie:** Ciascuna foglia corrisponde alla terminazione del processo di calcolo e racchiude la risposta o configurazione risolutiva finale del problema (ad esempio, la corretta permutazione degli indici che ordina l'array, o l'indice in cui risiede la chiave cercata).

In Figura 4.1 viene illustrata la topologia generale di un albero di decisione binario per un generico problema basato su confronti.

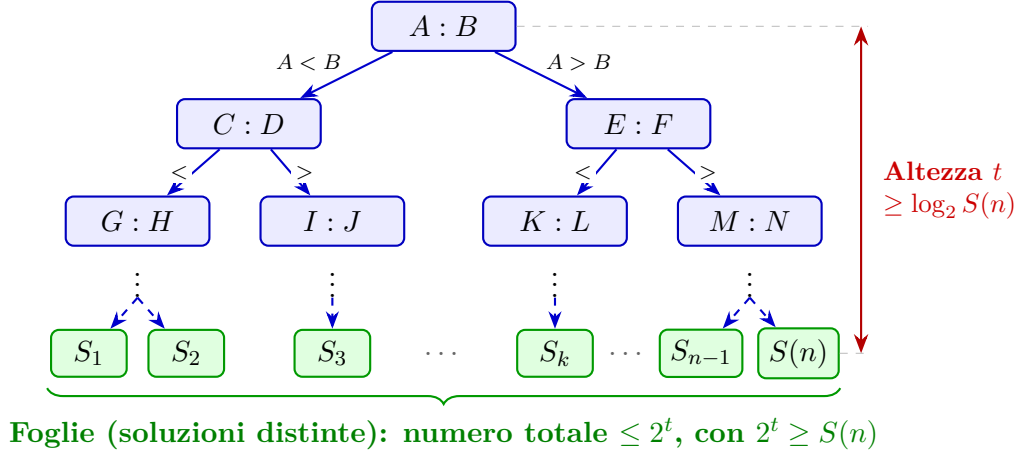


Figura 4.1: Modello dell'albero di decisione binario per algoritmi basati su confronti. I nodi interni rappresentano i confronti elementari; le foglie rappresentano le soluzioni distinte ammissibili ($S_1, S_2, \dots, S(n)$). Il cammino più lungo determina il tempo nel caso peggiore.

4.1.2 Il Teorema Generale dell'Informazione

Il legame tra il numero di soluzioni distinte di un problema e il numero minimo di confronti necessari a risolverlo è sancito dal seguente teorema fondamentale della teoria dell'informazione applicata agli algoritmi.

Teorema 4.1.1 (Teorema Generale dell'Informazione per Alberi di Decisione). *Sia P un problema computazionale risolubile tramite un algoritmo deterministico basato su confronti. Se per un input di dimensione n il problema ammette $S(n)$ soluzioni distinte (ciascuna delle quali deve poter essere restituita dall'algoritmo per opportune istanze dell'input), allora qualsiasi algoritmo deterministico basato su confronti richiede nel caso peggiore un tempo:*

$$T(n) = \Omega(\log_2 S(n)). \quad (4.2)$$

Dimostrazione. Consideriamo l'albero di decisione binario associato a un qualsiasi algoritmo deterministico corretto per il problema P . Sia t l'altezza dell'albero, ovvero la lunghezza massima di un cammino dalla radice a una foglia.

Poiché a ogni passo il confronto è binario (avendo assunto chiavi distinte), ogni nodo interno possiede al più 2 figli. Per induzione sull'altezza, un albero

binario di altezza t possiede al massimo un numero di foglie L pari a:

$$L \leq 2^t. \quad (4.3)$$

Affinché l'algoritmo sia corretto su qualsiasi istanza ammissibile, l'albero di decisione deve essere in grado di produrre ciascuna delle $S(n)$ possibili risposte distinte. Ne consegue che ogni soluzione distinta deve essere associata ad almeno una foglia distinta dell'albero (se due soluzioni distinte convergessero sulla medesima foglia, l'algoritmo fallirebbe nel distinguerle, restituendo un risultato errato per almeno una delle istanze corrispondenti). Pertanto, deve necessariamente valere la disuguaglianza:

$$2^t \geq L \geq S(n). \quad (4.4)$$

Applicando la funzione logaritmica in base 2 (strettamente monotona crescente) ad ambo i membri, si ottiene:

$$t \geq \log_2 S(n). \quad (4.5)$$

Poiché il tempo di esecuzione nel caso peggiore $T(n)$ coincide per definizione con l'altezza t dell'albero, si conclude che:

$$T(n) \geq \log_2 S(n) \implies T(n) = \Omega(\log_2 S(n)). \quad (4.6)$$

□

4.1.3 Applicazioni Fondamentali: Ricerca e Ordinamento

Il [Teorema 4.1.1](#) consente di ricavare immediatamente i limiti inferiori canonici per due problemi cardine della computazione in memoria interna:

1. **Ricerca di una chiave (*Searching*):** Dato un array ordinato di n elementi e una chiave cercata K , l'algoritmo deve stabilire se K è presente nell'array e, in caso affermativo, restituire l'indice $i \in \{1, 2, \dots, n\}$ corrispondente; se la chiave è assente, restituisce un simbolo speciale di fallimento $\perp$. L'insieme delle possibili risposte distinte è dunque:

$$S(n) = \{1, 2, \dots, n\} \cup \{\perp\} \implies |S(n)| = n + 1. \quad (4.7)$$

Applicando il teorema:

$$T_{\text{Search}}(n) \geq \log_2(n + 1) = \Omega(\log_2 n). \quad (4.8)$$

Questo limite inferiore attesta l'ottimalità asintotica della *Ricerca Binaria* (*Binary Search*), la quale opera esattamente in $\Theta(\log_2 n)$ confronti.

2. **Ordinamento (*Sorting*):** Dato un vettore di n elementi distinti, ordinare l'array equivale a identificare quale tra tutte le possibili permutazioni degli indici sia quella che dispone le chiavi in ordine non decrescente. Poiché l'insieme $\{1, 2, \dots, n\}$ ammette esattamente $n!$ permutazioni distinte, il numero di soluzioni possibili corrisponde a:

$$S(n) = n!. \quad (4.9)$$

Applicando il teorema dei limiti inferiori:

$$T_{\text{Sort}}(n) \geq \log_2(n!). \quad (4.10)$$

Per esprimere questo valore in forma asintotica esplicita, ricorriamo all'**approssimazione di Stirling** per il fattoriale:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \implies \ln(n!) = n \ln n - n + \mathcal{O}(\ln n). \quad (4.11)$$

Convertendo il logaritmo in base 2:

$$\log_2(n!) = n \log_2 n - n \log_2 e + \Theta(\log_2 n) = \Theta(n \log_2 n). \quad (4.12)$$

Alternativamente, mediante una limitazione inferiore elementare per metà dei termini:

$$n! \geq \prod_{i=\lceil n/2 \rceil}^n i \geq \left(\frac{n}{2}\right)^{n/2}, \quad (4.13)$$

$$\log_2(n!) \geq \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} (\log_2 n - 1) = \Omega(n \log_2 n). \quad (4.14)$$

Si conclude che:

$$T_{\text{Sort}}(n) = \Omega(n \log_2 n). \quad (4.15)$$

Qualsiasi algoritmo deterministico o randomizzato basato su confronti (come *Merge Sort*, *Heap Sort* o il tempo atteso di *Quicksort*) non può scendere al di sotto della soglia asintotica $\Omega(n \log_2 n)$ nel modello RAM.

4.2 Modello a Due Livelli: Limite Inferiore di Aggarwal-Vitter

Nel modello di memoria a due livelli di Aggarwal e Vitter [2], il costo di esecuzione non è misurato dai confronti elementari della CPU, bensì dal numero t di operazioni di I/O, ciascuna delle quali trasferisce un intero blocco di dimensione B tra il disco non volatile e la memoria interna (RAM) di capacità pari a M elementi.

4.2.1 La Natura dell'I/O e i Confronti Gratuiti in Memoria Interna

L'analisi tradizionale dell'albero di decisione binario non è direttamente applicabile al calcolo degli I/O:

- Una singola operazione di lettura dal disco carica contemporaneamente B elementi in memoria rapida.
- Una volta che questi B elementi risiedono nella RAM accanto agli altri $M - B$ elementi già presenti, la CPU può eseguire **un numero arbitrariamente elevato di confronti interni a costo zero di I/O**.
- Tali confronti interni "gratuiti" consentono di riordinare istantaneamente tutti gli M elementi residenti in RAM, restringendo drasticamente il ventaglio delle permutazioni globali ancora compatibili con lo stato dei dati.

Pertanto, per determinare il lower bound su memoria esterna, è necessario costruire un **Albero di Decisione di I/O**, in cui ogni nodo rappresenta un'operazione di I/O (lettura di un blocco di taglia B) e gli archi uscenti corrispondono a tutte le possibili sistemazioni informative derivanti dall'interazione tra i B elementi letti e gli $M - B$ elementi già presenti in memoria interna.

4.2.2 Calcolo del Grado di Diramazione (Fan-out) dell'Albero di I/O

Quando un blocco di B elementi viene trasferito dal disco alla memoria interna (che accoglie M elementi totali, di cui $M - B$ preesistenti e B appena caricati), distinguiamo due scenari operativi:

1. **Lettura di un "Nuovo Blocco" (*New Block*):** Il blocco viene letto dal disco per la primissima volta: gli elementi in esso contenuti non sono mai stati osservati in precedenza da alcun'altra operazione dell'algoritmo.
 - L'ordine reciproco dei B elementi è totalmente ignoto: essi possono presentarsi internamente in una qualsiasi delle $B!$ permutazioni relative possibili.
 - Una volta caricati in RAM, i B nuovi elementi devono essere posizionati reciprocamente rispetto agli $M - B$ elementi già residenti,

determinando quale sia la loro collocazione ordinata comune nell'insieme totale di M elementi. Il numero di modi in cui B elementi indistinguibili nella loro posizione possono intercalarsi tra gli M slot totali è dato dal coefficiente binomiale:

$$\binom{M}{B} = \frac{M!}{B!(M-B)!}. \quad (4.16)$$

Moltiplicando le $B!$ configurazioni d'ordine interno per i $\binom{M}{B}$ modi di intercalamento, il fattore di diramazione (*fan-out*) X generato dalla lettura di un nuovo blocco risulta:

$$X = (B!) \cdot \binom{M}{B}. \quad (4.17)$$

2. **Rilettura di un "Vecchio Blocco" (*Old Block*):** Il blocco era già stato caricato, ordinato internamente e successivamente scritto su disco in una fase precedente della computazione (si pensi a un run generato durante un passo di merge).

- L'ordine relativo interno dei suoi B elementi è già integralmente noto alla computazione; non si ottiene pertanto alcuna nuova informazione sul loro ordine intrinseco (fattore 1 al posto di $B!$).
- L'unico incremento informativo risiede nell'intercalamento dei B elementi con gli altri $M - B$ elementi attualmente residenti in memoria interna.

Il fattore di diramazione (*fan-out*) Y per la rilettura di un vecchio blocco è quindi:

$$Y = \binom{M}{B}. \quad (4.18)$$

In [Figura 4.2](#) viene schematizzata la biforcazione logica e la cardinalità delle diramazioni per le due tipologie di trasferimento I/O.

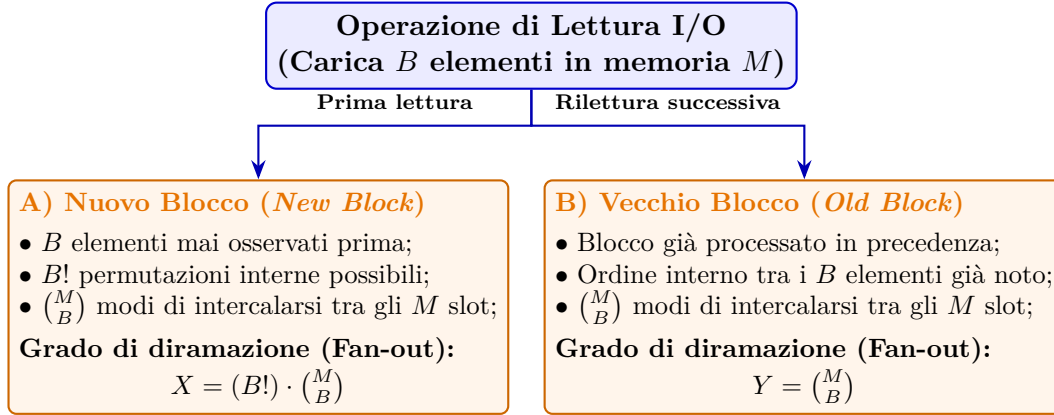


Figura 4.2: Fattori di diramazione (*fan-out*) nell'albero di decisione di I/O nel modello a due livelli: la lettura di un nuovo blocco apporta informazione d'ordine interno ($B!$) e di intercalamento ($\binom{M}{B}$); la riletture di un vecchio blocco apporta esclusivamente informazione di intercalamento.

4.2.3 Derivazione Formale del Lower Bound di Aggarwal-Vitter

Sia dato un file di ingresso composto da n elementi distinti suddivisi in $N = \frac{n}{B}$ blocchi su disco. Per poter ordinare il file, qualsiasi algoritmo corretto deve necessariamente ispezionare ciascun elemento almeno una volta: ne discende che l'algoritmo deve compiere almeno $\frac{n}{B}$ operazioni di lettura di tipo "Nuovo Blocco".

Se l'algoritmo esegue complessivamente t operazioni di I/O nel caso peggiore, allora:

- Esattamente $\frac{n}{B}$ operazioni saranno di tipo "Nuovo Blocco", ciascuna con fattore di diramazione al più X .
- Le rimanenti $t - \frac{n}{B}$ operazioni saranno (al più) di tipo "Vecchio Blocco", ciascuna con fattore di diramazione al più Y .

Il numero totale di foglie dell'albero di decisione di I/O rappresenta il numero massimo di permutazioni globali distinguibili dalla sequenza di trasferimenti. Affinché l'algoritmo sia in grado di ordinare correttamente qualsiasi istanza, tale numero di foglie deve risultare almeno pari al numero totale di permutazioni ammissibili $n!$:

$$X^{\frac{n}{B}} \cdot Y^{(t - \frac{n}{B})} \geq n!. \quad (4.19)$$

Sostituendo le espressioni di X ed Y definite nelle equazioni (4.17) e (4.18):

$$\left[(B!) \cdot \binom{M}{B} \right]^{\frac{n}{B}} \cdot \left[\binom{M}{B} \right]^{t - \frac{n}{B}} \geq n!. \quad (4.20)$$

Raccogliendo le potenze relative alla base comune $\binom{M}{B}$:

$$(B!)^{\frac{n}{B}} \cdot \binom{M}{B}^{\frac{n}{B}} \cdot \binom{M}{B}^{t - \frac{n}{B}} \geq n! \implies (B!)^{\frac{n}{B}} \cdot \binom{M}{B}^t \geq n!. \quad (4.21)$$

Applichiamo il logaritmo in base 2 a entrambi i membri della disuguaglianza (4.21):

$$\log_2 \left((B!)^{\frac{n}{B}} \right) + \log_2 \left(\binom{M}{B}^t \right) \geq \log_2(n!), \quad (4.22)$$

da cui:

$$\frac{n}{B} \log_2(B!) + t \log_2 \left(\binom{M}{B} \right) \geq \log_2(n!). \quad (4.23)$$

Per sviluppare i termini logaritmici utilizziamo le consuete approssimazioni asintotiche per il fattoriale e per il coefficiente binomiale:

- Per l'approssimazione di Stirling: $\log_2(a!) \approx a \log_2 a - a \log_2 e \approx a \log_2 a$.
- Per il coefficiente binomiale, ricordando che $\binom{a}{b} \leq \left(\frac{e \cdot a}{b} \right)^b$:

$$\log_2 \left(\binom{M}{B} \right) \approx B \log_2 \left(\frac{M}{B} \right) + \mathcal{O}(B). \quad (4.24)$$

Sostituendo queste espressioni nell'equazione (4.23):

$$\frac{n}{B} \cdot (B \log_2 B) + t \cdot \left(B \log_2 \frac{M}{B} \right) \geq n \log_2 n. \quad (4.25)$$

Semplificando il fattore B nel primo termine:

$$n \log_2 B + t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n. \quad (4.26)$$

Isoliamo il termine contenente l'incognita t :

$$t \cdot B \log_2 \left(\frac{M}{B} \right) \geq n \log_2 n - n \log_2 B = n (\log_2 n - \log_2 B) = n \log_2 \left(\frac{n}{B} \right). \quad (4.27)$$

Dividendo ambo i membri per la quantità strettamente positiva $B \log_2 \left(\frac{M}{B} \right)$:

$$t \geq \frac{n \log_2 \left(\frac{n}{B} \right)}{B \log_2 \left(\frac{M}{B} \right)} = \frac{n}{B} \cdot \frac{\log_2(n/B)}{\log_2(M/B)}. \quad (4.28)$$

Ricorrendo alla proprietà del cambio di base dei logaritmi ($\frac{\log_2 a}{\log_2 b} = \log_b a$), otteniamo formalmente il celebre teorema di Aggarwal e Vitter.

Teorema 4.2.1 (Limite Inferiore di Aggarwal-Vitter per l'Ordinamento). *Nel modello di memoria a due livelli, qualsiasi algoritmo deterministico o randomizzato basato su confronti richiede nel caso peggiore un numero di trasferimenti I/O pari a:*

$$I/O_{Sort}(n) = \Omega \left(\frac{n}{B} \log_{M/B} \left(\frac{n}{B} \right) \right) = \Omega \left(\frac{n}{B} \frac{\log(n/B)}{\log(M/B)} \right). \quad (4.29)$$

Proprietà 4.2.1 (Ottimalità Asintotica del Multiway Merge Sort). *Il limite inferiore di Aggarwal e Vitter dimostra che il **Multiway Merge Sort** (k -Way Merge Sort con fattore di merge $k = \frac{M}{B} - 1$) analizzato nella Lezione 3 è **asintoticamente ottimo**: il numero di I/O spesi per ordinare l'array coincide esattamente con la barriera teorica insuperabile imposta dalla teoria dell'informazione su memoria esterna.*

4.3 Quicksort: Gestione dei Duplicati e Dual-Pivot Quicksort

Quicksort è un algoritmo basato sul paradigma *Divide et Impera* di tipo distributivo (*distribution-based*): anziché dividere rigidamente l'array per posizione (come in *Merge Sort*) per poi unire i risultati ordinati, Quicksort partiziona i dati in base al valore di una o più chiavi privilegiate denominate **pivot**, delegando l'intero lavoro di ordinamento alla fase di distribuzione (*partition*).

4.3.1 Il Problema dei Valori Uguali (Equal Values)

Nelle implementazioni didattiche standard di Quicksort basate sul partizionamento a due vie (*2-way partitioning*, schema di Lomuto o di Hoare):

- L'array viene ripartito in due sole categorie: elementi $\leq p$ da una parte ed elementi $> p$ dall'altra.

- Qualora l'array contenga una frequenza elevata di elementi identici al pivot p (nel caso limite, un array composto esclusivamente da elementi tutti uguali, $S[i] = c, \forall i$), tale schema inserisce tutti gli elementi duplicati all'interno della medesima partizione.
- La dimensione della sotto-istanza si riduce di una sola unità a ogni invocazione (1 contro $n - 1$), degradando l'albero di ricorsione a un albero degenero di altezza $\Theta(n)$ e il tempo complessivo di calcolo alla complessità pessima:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2). \quad (4.30)$$

Definizione 4.3.1 (Partizionamento a Tre Vie (Dutch National Flag)). La soluzione classica al problema dei duplicati, formulata da Edsger Dijkstra [3] con il problema della bandiera olandese, consiste nel partizionare l'array in **tre zone contigue**:

$$[< p \quad | \quad = p \quad | \quad > p]. \quad (4.31)$$

Gli elementi strettamente uguali al pivot ($= p$) si trovano già nella loro collocazione definitiva e vengono **esclusi dalle successive chiamate ricorsive**, le quali vengono lanciate esclusivamente sulle fasce $[< p]$ e $[> p]$. Su array con elementi tutti uguali, l'algoritmo converge in tempo strettamente lineare $\mathcal{O}(n)$.

4.3.2 Algoritmo di Partizionamento a 3 Vie ed Esempio Esecutivo

Per implementare concretamente il partizionamento a tre vie (*Dutch National Flag*) in Quicksort, si adotta comunemente lo schema lineare ideato da Dijkstra e formalizzato da Sedgwick [4].

Dato un sotto-array $S[\text{lo} \dots \text{hi}]$ e scelto come pivot l'elemento iniziale $p = S[\text{lo}]$, l'algoritmo mantiene tre indici operativi durante la scansione:

- **Puntatore lt (*less than*)**: delimita a sinistra gli elementi strettamente inferiori al pivot. L'invariante garantisce che $S[k] < p$ per ogni $\text{lo} \leq k < lt$. Inizialmente $lt = \text{lo}$.
- **Puntatore i (cursore di scansione)**: scorre gli elementi dell'array. L'invariante assicura che $S[k] = p$ per ogni $lt \leq k < i$. Inizialmente $i = \text{lo} + 1$.
- **Puntatore gt (*greater than*)**: delimita a destra gli elementi strettamente superiori al pivot. L'invariante garantisce che $S[k] > p$ per ogni $gt < k \leq \text{hi}$. Inizialmente $gt = \text{hi}$.

- L'intervallo $S[i \dots gt]$ contiene gli elementi ancora non esaminati.

A ogni iterazione, finché $i \leq gt$, si confronta l'elemento corrente $S[i]$ con il pivot p :

1. **Se** $S[i] < p$: si scambia $S[lt]$ con $S[i]$, quindi si incrementano entrambi gli indici ($lt \leftarrow lt + 1, i \leftarrow i + 1$). Poiché per invariante la cella $S[lt]$ conteneva un elemento $= p$, lo scambio sposta il valore minore a sinistra e posiziona un duplicato del pivot in $S[i]$, consentendo a i di avanzare.
2. **Se** $S[i] > p$: si scambia $S[i]$ con $S[gt]$, quindi si decrementa solo gt ($gt \leftarrow gt - 1$). L'indice i **non avanza**, poiché l'elemento appena scambiato da $S[gt]$ non è ancora stato esaminato.
3. **Se** $S[i] == p$: l'elemento si trova già nella collocazione centrale corretta; si incrementa unicamente il cursore di scansione ($i \leftarrow i + 1$).

Al termine della procedura ($i > gt$), l'array risulta ripartito nelle tre zone: $S[lo \dots lt - 1] < p$, $S[lt \dots gt] = p$ e $S[gt + 1 \dots hi] > p$. Le successive chiamate ricorsive vengono invocate **esclusivamente** su $S[lo \dots lt - 1]$ e su $S[gt + 1 \dots hi]$, escludendo l'intero blocco di duplicati.

In [Figura 4.3](#) viene illustrato lo schema dell'invariante e la traccia visiva su un array di esempio con pivot $p = 4$.

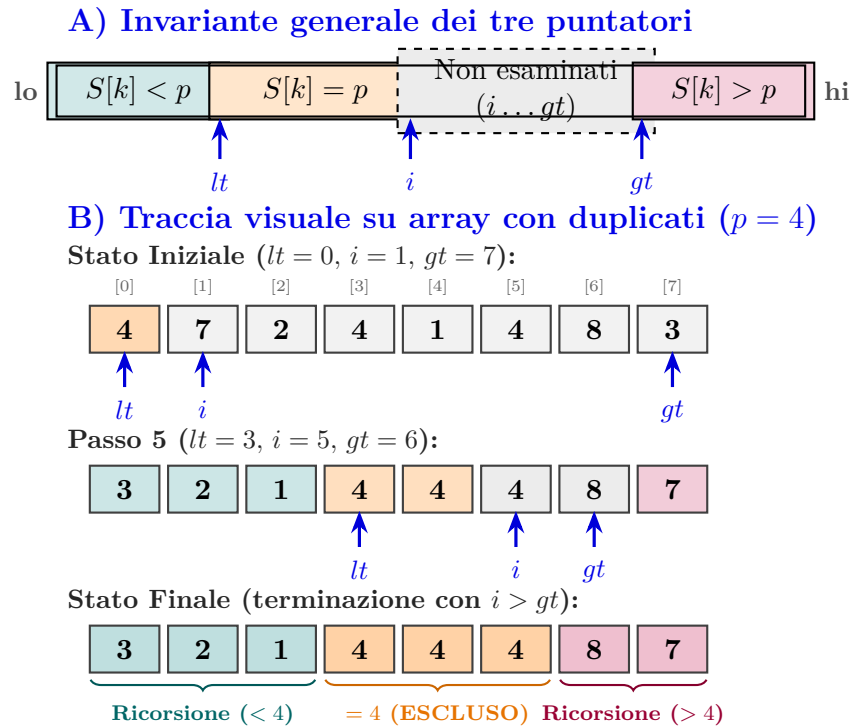


Figura 4.3: Partizionamento a tre vie di Dijkstra (*Dutch National Flag*): A) Invariante dei tre puntatori lt, i, gt . B) Traccia visiva dell'evoluzione dell'array per pivot $p = 4$. La fascia centrale contenente tutti i duplicati del pivot è collocata nella sua posizione definitiva ed esclusa dalle chiamate ricorsive.

In Tabella 4.1 viene riportata l'evoluzione dettagliata passo per passo dei tre puntatori e del contenuto dell'array durante l'esecuzione dell'algoritmo.

Passo	i	lt	gt	Confronto	Azione	Stato Array
Init	1	0	7	Pivot $p = 4$	Inizializzazione	[4, 7, 2, 4, 1, 4, 8, 3]
1	1	0	7	$S[1] = 7 > 4$	SWAP($S[1], S[7]$), $gt \leftarrow 6$	[4, 3, 2, 4, 1, 4, 8, 7]
2	1	0	6	$S[1] = 3 < 4$	SWAP($S[0], S[1]$), $lt \leftarrow 1, i \leftarrow 2$	[3, 4, 2, 4, 1, 4, 8, 7]
3	2	1	6	$S[2] = 2 < 4$	SWAP($S[1], S[2]$), $lt \leftarrow 2, i \leftarrow 3$	[3, 2, 4, 4, 1, 4, 8, 7]
4	3	2	6	$S[3] == 4$	Avanzamento $i \leftarrow 4$	[3, 2, 4, 4, 1, 4, 8, 7]
5	4	2	6	$S[4] = 1 < 4$	SWAP($S[2], S[4]$), $lt \leftarrow 3, i \leftarrow 5$	[3, 2, 1, 4, 4, 4, 8, 7]
6	5	3	6	$S[5] == 4$	Avanzamento $i \leftarrow 6$	[3, 2, 1, 4, 4, 4, 8, 7]
7	6	3	6	$S[6] = 8 > 4$	SWAP($S[6], S[6]$), $gt \leftarrow 5$	[3, 2, 1, 4, 4, 4, 8, 7]
Fine	6	3	5	$i > gt$	Terminazione	[3, 2, 1 4, 4, 4 8, 7]

Tabella 4.1: Traccia puntuale passo-passo del partizionamento a 3 vie con pivot $p = 4$. Al termine, i tre elementi identici 4 occupano gli indici $lt \dots gt$ (3...5) e vengono esclusi dalle chiamate ricorsive.

4.3.3 Architettura del Dual-Pivot Quicksort (Yaroslavskiy)

Nel 2009, l'ingegnere Vladimir Yaroslavskiy propose un algoritmo innovativo denominato **Dual-Pivot Quicksort** [5]. Dopo intensi benchmark empirici condotti in collaborazione con Joshua Bloch e Robert Sedgewick, l'algoritmo ha sostituito il Quicksort classico a pivot singolo nella libreria standard di Java ('java.util.Arrays.sort') per tutti i tipi primitivi.

A differenza del Quicksort classico, il Dual-Pivot Quicksort seleziona **due pivot** P e Q . Inizialmente si verifica che $P \leq Q$ (scambiandoli se necessario). L'intervallo dell'array $S[\text{inizio} \dots \text{fine}]$ viene suddiviso dinamicamente in quattro regioni gestite tramite tre indici operativi (l, k, g):

- l (*less*): limite superiore degli elementi strettamente minori di P ($S[i] < P$ per ogni inizio $\leq i < l$).
- k : indice di scansione corrente ($l \leq k \leq g$). Gli elementi tra l e $k - 1$ soddisfano l'invariante centrale: $P \leq S[i] \leq Q$.

- g (*great*): limite inferiore degli elementi strettamente maggiori di Q ($S[i] > Q$ per ogni $g < i \leq \text{fine}$).
- La regione compresa tra k e g rappresenta l'insieme degli elementi ancora non esaminati.

In Figura 4.4 viene illustrato lo stato dell'array durante l'esecuzione del partizionamento a due pivot.

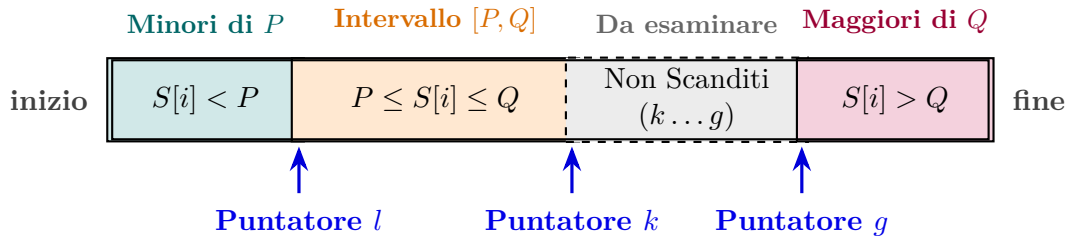


Figura 4.4: Schema del partizionamento nel Dual-Pivot Quicksort di Yaroslavskiy. L'array è partizionato nelle quattro regioni dinamiche: elementi $< P$, elementi compresi tra P e Q , elementi non ancora esaminati tra k e g , ed elementi $> Q$.

4.3.4 Invariante e Logica di Avanzamento

La procedura di partizionamento itera mantenendo l'invariante fino a quando l'indice di scansione k non supera l'indice g ($k \leq g$). A ogni iterazione viene esaminato il valore corrente $S[k]$:

1. **Caso 1:** $S[k] < P$: L'elemento appartiene alla prima partizione. Si effettua uno scambio tra $S[k]$ e $S[l]$, dopodiché si incrementano entrambi i puntatori:

$$\text{SWAP}(S[k], S[l]), \quad l \leftarrow l + 1, \quad k \leftarrow k + 1. \quad (4.32)$$

Poiché per invariante l'elemento preesistente in $S[l]$ apparteneva alla fascia centrale $[P, Q]$, dopo lo scambio la posizione k conterrà un valore valido per la seconda partizione, consentendo a k di avanzare.

2. **Caso 2:** $S[k] > Q$: L'elemento appartiene alla partizione superiore. Prima dello scambio, si fa retrocedere il puntatore g finché non si individua una posizione tale che $S[g] \leq Q$ (saltando elementi già corretti per la parte destra). Trovato tale g , si scambia:

$$\text{SWAP}(S[k], S[g]), \quad g \leftarrow g - 1. \quad (4.33)$$

L'elemento collocato in $S[k]$ (proveniente da g) non è ancora stato classificato rispetto a P : esso viene pertanto **riesaminato al passo successivo senza avanzare k** .

3. **Caso 3:** $P \leq S[k] \leq Q$: L'elemento risiede già nella collocazione corretta della fascia centrale. Non si rende necessario alcuno scambio fisico di memoria e l'indice di scansione avanza semplicemente:

$$k \leftarrow k + 1. \quad (4.34)$$

Al termine della scansione ($k > g$), i due pivot P e Q vengono ricollocati scambiandoli rispettivamente nelle loro posizioni definitive $l - 1$ e $g + 1$. Le tre chiamate ricorsive vengono quindi eseguite in modo indipendente sui tre intervalli:

$$[\text{inizio} \dots l - 2], \quad [l \dots g], \quad [g + 2 \dots \text{fine}]. \quad (4.35)$$

Algorithm 3: Dual-Pivot Partitioning (Yaroslavskiy)

Dati: Array S , indice iniziale $left$, indice finale $right$ con $left < right$

Risultato: Indici di collocazione finale dei pivot (p_1, p_2)

```
1 if  $S[left] > S[right]$  then
2    $\lfloor$  SWAP( $S[left], S[right]$ )
3    $P \leftarrow S[left]$ 
4    $Q \leftarrow S[right]$ 
5    $l \leftarrow left + 1$ 
6    $g \leftarrow right - 1$ 
7    $k \leftarrow l$ 
8   while  $k \leq g$  do
9     if  $S[k] < P$  then
10      SWAP( $S[k], S[l]$ )
11       $l \leftarrow l + 1$ 
12       $k \leftarrow k + 1$ 
13     else if  $S[k] > Q$  then
14       while  $S[g] > Q$  and  $k < g$  do
15          $g \leftarrow g - 1$ 
16       SWAP( $S[k], S[g]$ )
17        $g \leftarrow g - 1$ 
18       if  $S[k] < P$  then
19         SWAP( $S[k], S[l]$ )
20          $l \leftarrow l + 1$ 
21        $k \leftarrow k + 1$ 
22     else
23        $k \leftarrow k + 1$ 
24    $l \leftarrow l - 1$ 
25    $g \leftarrow g + 1$ 
26   SWAP( $S[left], S[l]$ )
27   SWAP( $S[right], S[g]$ )
28 return  $(l, g)$ 
```

4.3.5 Esempio Esecutivo e Traccia del Dual-Pivot Quicksort

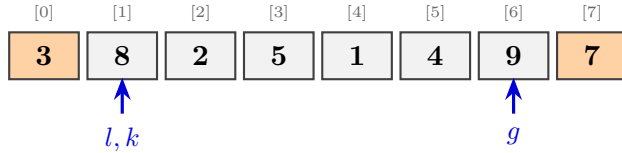
Per illustrare il funzionamento del partizionamento a due pivot descritto nell'[algorithm 3](#), consideriamo un array di 8 elementi:

$$S = [3, 8, 2, 5, 1, 4, 9, 7]. \quad (4.36)$$

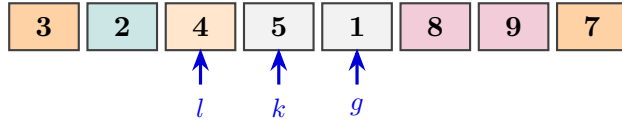
I pivot vengono scelti agli estremi dell'array: $P = S[0] = 3$ e $Q = S[7] = 7$. Poiché $P \leq Q$ ($3 \leq 7$), l'ordinamento preliminare dei pivot è già soddisfatto. Gli indici iniziali vengono configurati come: $l = 1$, $k = 1$ e $g = 6$.

In [Figura 4.5](#) vengono illustrati i passaggi salienti dell'esecuzione, evidenziando il movimento dei cursori l, k, g e la ricollocazione finale di P e Q .

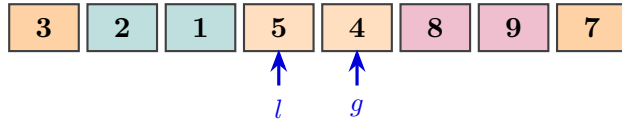
Stato Iniziale ($P = 3, Q = 7, l = 1, k = 1, g = 6$):



Passo 2 ($S[2] = 2 < 3 \implies \text{SWAP}(S[1], S[2]), l \leftarrow 2, k \leftarrow 3$):



Fine Scansione ($k = 5 > g = 4$, partizioni isolate prima del posizionamento dei pivot):



Stato Finale (scambio pivot con $l - 1$ e $g + 1$, ripartizione a 3 vie):

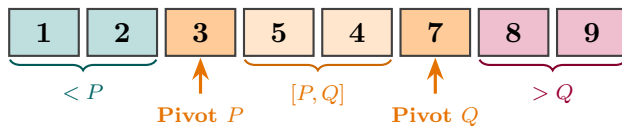


Figura 4.5: Traccia dell'esecuzione del partizionamento Dual-Pivot (schema di Yaroslavskiy) su un array con $P = 3$ e $Q = 7$. Dopo la scansione, i pivot vengono scambiati nelle rispettive posizioni definitive (indici 2 e 5), isolando le tre partizioni su cui proseguirà la ricorsione.

In [Tabella 4.2](#) viene riassunta la sequenza puntuale delle operazioni compiute dal Dual-Pivot Quicksort.

Passo	k	l	g	Elemento	Azione	Stato Array
Init	1	1	6	Pivot $P = 3, Q = 7$	Inizializzazione cursori	[3, 8, 2, 5, 1, 4, 9, 7]
1	1	1	6	$S[1] = 8 > 7$	Trova $g = 5$; SWAP($S[1], S[5]$), $g \leftarrow 4, k \leftarrow 2$	[3, 4, 2, 5, 1, 8, 9, 7]
2	2	1	4	$S[2] = 2 < 3$	SWAP($S[1], S[2]$), $l \leftarrow 2, k \leftarrow 3$	[3, 2, 4, 5, 1, 8, 9, 7]
3	3	2	4	$3 \leq S[3] = 5 \leq 7$	Avanzamento $k \leftarrow 4$	[3, 2, 4, 5, 1, 8, 9, 7]
4	4	2	4	$S[4] = 1 < 3$	SWAP($S[2], S[4]$), $l \leftarrow 3, k \leftarrow 5$	[3, 2, 1, 5, 4, 8, 9, 7]
Fine	5	3	4	$k > g$	Arresto scansione	[3, 2, 1, 5, 4, 8, 9, 7]
Pivot	–	2	5	Ricollocazione	SWAP($S[0], S[2]$), SWAP($S[7], S[5]$)	[1, 2 3 5, 4 7 8, 9]

Tabella 4.2: Traccia puntuale passo-passo del partizionamento Dual-Pivot con pivot $P = 3$ e $Q = 7$. Al termine, i due pivot sono collocati nelle posizioni definitive (indici 2 e 5) e le chiamate ricorsive procedono separatamente sulle tre sotto-regioni.

Proprietà 4.3.1 (Vantaggi Architetture del Dual-Pivot Quicksort). *L'adozione di due pivot comporta significativi benefici a livello di ingegneria algoritmica:*

- **Minore profondità dell'albero di ricorsione:** *Suddividendo lo spazio in 3 parti anziché in 2, l'altezza media dell'albero di ricorsione si riduce da $\log_2 n \approx 1.44 \ln n$ a $\log_3 n \approx 0.91 \ln n$, determinando una riduzione del numero complessivo di scansioni della memoria.*
- **Efficienza di Cache e CPU pipelining:** *Sebbene il numero teorico di confronti per elemento aumenti leggermente rispetto al Quicksort classico ottimizzato ($1.9n \ln n$ contro $2n \ln n$), il numero di scritture e di cache miss si riduce marcatamente, sfruttando al massimo la banda di trasferimento della gerarchia di memoria.*

4.4 Bounded Quicksort (BQS) ed Eliminazione della Tail Recursion

Nonostante l'eccellente tempo atteso $\mathcal{O}(n \log_2 n)$, l'implementazione ricorsiva ordinaria di Quicksort presenta una grave criticità legata al consumo di

memoria ausiliaria sullo stack di esecuzione (*call stack*).

4.4.1 La Criticità dello Stack di Ricorsione

A ogni chiamata ricorsiva, il sistema operativo alloca un frame di stack contenente i parametri della funzione, le variabili locali e l'indirizzo di ritorno:

- Nel caso ottimo o bilanciato, lo stack raggiunge una profondità pari a $\Theta(\log_2 n)$, richiedendo uno spazio ausiliario trascurabile.
- Tuttavia, qualora il partizionamento risulti costantemente sbilanciato (caso peggiore: pivot posizionato all'estremo 1 contro $n-1$), la profondità dello stack cresce linearmente con la dimensione dell'input:

$$\text{Profondità Stack} = \Theta(n). \quad (4.37)$$

Per istanze di grandi dimensioni ($n \geq 10^6$), una profondità di stack lineare genera inevitabilmente l'interruzione anomala del programma per esaurimento dello stack (*Stack Overflow Exception*).

4.4.2 Algoritmo Bounded Quicksort (BQS)

Il **Bounded Quicksort (BQS)** risolve questa vulnerabilità strutturale attraverso due tecniche cardine dell'ingegneria del software:

1. **Eliminazione della Ricorsione in Coda (*Tail Call Elimination*):** l'algoritmo non esegue mai due chiamate ricorsive all'interno della stessa funzione. Esegue la ricorsione **esclusivamente sulla partizione di dimensione inferiore o uguale alla metà dell'intervallo corrente** ($\leq \frac{j-i}{2}$). La partizione rimanente (la più grande) viene elaborata in modo puramente iterativo aggiornando gli indici i o j all'interno di un ciclo 'while'.
2. **Cutoff ad Insertion Sort su blocchi piccoli:** quando l'ampiezza dell'intervallo scende al di sotto di una soglia predefinita m_0 (tipicamente $m_0 \in [16, 32]$), la ricorsione viene interrotta a favore dell'ordinamento per inserzione.

Algorithm 4: Bounded Quicksort (BQS)

Dati: Array S , indice iniziale i , indice finale j , soglia m_0

Risultato: Sottoarray $S[i \dots j]$ ordinato

```
1 while  $(j - i) \geq m_0$  do
2    $r \leftarrow \text{RANDOM\_POSITION}(i, \dots, j)$ 
3    $\text{SWAP}(S[r], S[i])$  // Pivot casuale portato all'inizio
4    $p' \leftarrow \text{PARTITION}(S, i, j)$  //  $p'$  è la posizione finale del
      pivot
5   if  $p' \leq \lfloor (i + j)/2 \rfloor$  then
      // La partizione SINISTRA è la più piccola:
      ricorsione
6   BQS( $S, i, p' - 1, m_0$ )
      // Eliminazione tail recursion sulla DESTRA:
      iterazione
7    $i \leftarrow p' + 1$ 
8   else
      // La partizione DESTRA è la più piccola:
      ricorsione
9   BQS( $S, p' + 1, j, m_0$ )
      // Eliminazione tail recursion sulla SINISTRA:
      iterazione
10   $j \leftarrow p' - 1$ 
11 INSERTION_SORT( $S, i, j$ ) // Risoluzione blocchi piccoli
    di dimensione  $< m_0$ 
```

In [Figura 4.6](#) viene illustrato il diagramma di flusso del controllo operativo di BQS, mettendo in risalto la diramazione tra ricorsione vincolata e continuazione iterativa.

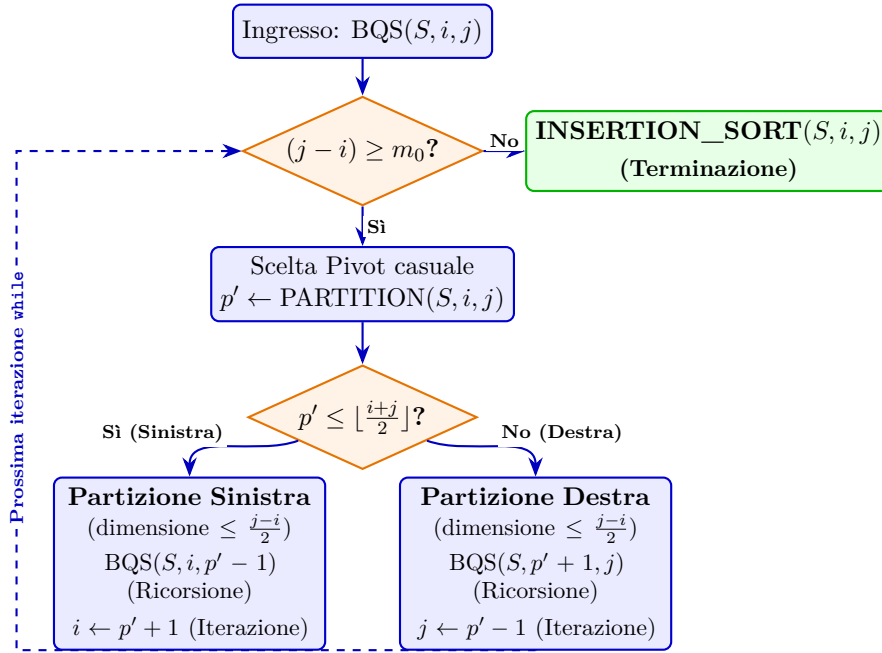


Figura 4.6: Diagramma di controllo del Bounded Quicksort (BQS). La chiamata ricorsiva viene lanciata rigidamente sulla metà di dimensione minore ($\leq (j - i)/2$), mentre la partizione più grande viene elaborata mediante iterazione continua nel ciclo `while`. I blocchi inferiori a m_0 sono ordinati direttamente da Insertion Sort.

4.4.3 Esempio Esecutivo e Gestione dello Stack nel Bounded Quicksort

Per comprendere a fondo il meccanismo di confinamento della profondità dello stack e l'eliminazione della ricorsione in coda (*tail recursion elimination*), consideriamo l'esecuzione dell'[algorithm 4](#) su un array di 8 elementi con soglia di cutoff $m_0 = 3$:

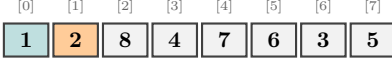
$$S = [7, 2, 8, 4, 1, 6, 3, 5]. \quad (4.38)$$

In uno scenario sfavorevole in cui il partizionamento risulta costantemente sbilanciato, un'implementazione ordinaria di Quicksort allocherebbe una sequenza lineare di frame sullo stack di chiamata, raggiungendo una profondità $\Theta(n)$. Al contrario, Bounded Quicksort garantisce che la partizione più voluminosa venga sempre elaborata all'interno dello stesso frame mediante continuazione ciclica (aggiornando i puntatori i o j), mentre la ricorsione viene invocata unicamente sulla partizione di dimensione non superiore alla metà ($\leq L/2$).

In Figura 4.7 vengono illustrati i passaggi esecutivi e il confronto strutturale della memoria dello stack tra Quicksort classico e Bounded Quicksort.

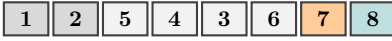
A. Sequenza delle Iterazioni nel Loop Principale di BQS ($m_0 = 3$)

Iterazione 1: Finestra $[0 \dots 7]$ ($L = 8$). Pivot $p' = 1$ (valore 2). Test: $p' = 1 \leq \lfloor 7/2 \rfloor = 3$ (Vero)



- Ricorsione su $[0 \dots 0]$ ($\dim = 1 \leq 4$, frame temporaneo)
- Iterazione su $[2 \dots 7]$ ($i \leftarrow p' + 1 = 2$, nello stesso frame 0)

Iterazione 2: Finestra $[2 \dots 7]$ ($L = 6$). Pivot $p' = 6$ (valore 7). Test: $p' = 6 \leq \lfloor 9/2 \rfloor = 4$ (Falso)



- Ricorsione su $[7 \dots 7]$ ($\dim = 1 \leq 3$, frame temporaneo)
- Iterazione su $[2 \dots 5]$ ($j \leftarrow p' - 1 = 5$, nello stesso frame 0)

Iterazione 3: Finestra $[2 \dots 5]$ ($L = 4$). Pivot $p' = 3$ (valore 4). Test: $p' = 3 \leq \lfloor 7/2 \rfloor = 3$ (Vero)



- Ricorsione su $[2 \dots 2]$ ($\dim = 1 \leq 2$, frame temporaneo)
- Iterazione su $[4 \dots 5]$ ($i \leftarrow p' + 1 = 4$, nello stesso frame 0)

Cutoff Base: Finestra $[4 \dots 5]$ ($L = 2 < m_0 = 3$). Uscita dal ciclo while



- **INSERTION_SORT**($S, 4, 5$) $\Rightarrow$ ordina $[6, 5] \rightarrow [5, 6]$
- L'intero array risulta ordinato: $[1, 2, 3, 4, 5, 6, 7, 8]$.

B. Struttura del Call Stack: Quicksort Standard vs Bounded Quicksort

Quicksort Classico (Stack Lineare $\Theta(n)$) **Bounded Quicksort (Stack Logaritmico $\leq \lfloor \log_2 n \rfloor$):**

Depth 3: $QS(S, 4, 5)$ (In esecuzione)
Depth 2: $QS(S, 2, 5)$ (In attesa)
Depth 1: $QS(S, 2, 7)$ (In attesa)
Depth 0: $QS(S, 0, 7)$ (Radice in attesa)

$\Rightarrow$ I frame si accumulano linearmente: con partizioni sbilanciate la profondità raggiunge $\Theta(n)$.

Depth 1: $BQS(\leq L/2)$ (Istantaneo, liberato subito)

Depth 0: $BQS(S, 0, 7)$ (Unico frame permanente)

- Iterazione 1: intervallo $[0, 7]$
- Iterazione 2: intervallo $[2, 7]$ (aggiornato i)
- Iterazione 3: intervallo $[2, 5]$ (aggiornato j)
- Cutoff m_0 : intervallo $[4, 5] \rightarrow$ Insertion Sort

$\Rightarrow$ La parte maggiore resta nel frame 0; la ricorsione su $\leq L/2$ non supera mai $\lfloor \log_2 n \rfloor$.

Figura 4.7: Traccia dell'esecuzione di Bounded Quicksort (BQS) con soglia $m_0 = 3$ e confronto della memoria di stack. **(A)** Ad ogni iterazione, la partizione sbilanciata maggiore viene mantenuta nello stesso stack frame aggiornando i o j , mentre solo la partizione dimezzata ($\leq L/2$) genera una breve chiamata ricorsiva. **(B)** Il Quicksort ordinario accumula frame lineari; BQS garantisce uno stack a profondità rigorosamente logaritmica.

In Tabella 4.3 viene dettagliata la sequenza puntuale delle iterazioni, evidenziando il test sulla partizione minima, l'azione di tail call elimination e la profondità massima istantanea dello stack di esecuzione.

Passo	$[i, j]$	L	Test Pivot	Azione (Ricorsione & Iterazione)	Stato Array	Stack
Init	$[0, 7]$	8	–	Frame radice BQS($S, 0, 7$)	$[7, 2, 8, 4, 1, 6, 3, 5]$	0
1	$[0, 7]$	8	$p' = 1 \leq 3$	Rec: $[0, 0]$ (dim ≤ 4 , istantanea) Iter: $i \leftarrow 2$ su $[2, 7]$ (stesso frame)	$[1 \mid \mathbf{2} \mid 8, 4, 7, 6, 3, 5]$	1
2	$[2, 7]$	6	$p' = 6 > 4$	Rec: $[7, 7]$ (dim ≤ 3 , istantanea) Iter: $j \leftarrow 5$ su $[2, 5]$ (stesso frame)	$[1, 2 \mid 5, 4, 3, 6 \mid \mathbf{7} \mid 8]$	1
3	$[2, 5]$	4	$p' = 3 \leq 3$	Rec: $[2, 2]$ (dim ≤ 2 , istantanea) Iter: $i \leftarrow 4$ su $[4, 5]$ (stesso frame)	$[1, 2 \mid 3 \mid \mathbf{4} \mid 6, 5 \mid 7 \mid 8]$	1
Cutoff	$[4, 5]$	2	$L < m_0$	Exit: Uscita dal ciclo while Sort: Insertion Sort su $[4, 5]$	$[1, 2, 3, 4, \mathbf{5}, \mathbf{6}, 7, 8]$	0

Tabella 4.3: Traccia dell'esecuzione passo-passo di Bounded Quicksort (BQS) con soglia di cutoff $m_0 = 3$. In ogni iterazione, la partizione di dimensione minore o uguale a $L/2$ viene risolta con una chiamata ricorsiva limitata, mentre la partizione più grande viene elaborata iterativamente all'interno del frame corrente, mantenendo la profondità dello stack ≤ 1 nell'esempio ($\leq \lfloor \log_2 n \rfloor$ nel caso generale).

4.4.4 Analisi Rigorosa dello Spazio Extra $\Theta(\log_2 n)$

L'aspetto fondamentale di BQS risiede nella garanzia assoluta sul consumo di memoria di sistema:

Teorema 4.4.1 (Garanzia dello Spazio Ausiliario di Bounded Quicksort). *Per qualsiasi input di n elementi, l'algoritmo Bounded Quicksort alloca uno spazio aggiuntivo sullo stack di sistema rigidamente limitato a:*

$$\text{Spazio Ausiliario} = \Theta(\log_2 n). \quad (4.39)$$

La profondità massima dello stack non eccede $\lfloor \log_2 n \rfloor$ nel caso peggiore assoluto.

Dimostrazione. Sia L la lunghezza della finestra corrente dell'array all'ingresso di una chiamata a BQS, definita da $L = j - i + 1$. Quando il pivot viene posizionato nell'indice p' , l'algoritmo effettua una diramazione esaminando la dimensione delle due partizioni:

- La partizione sinistra ha ampiezza $L_{\text{left}} = (p' - 1) - i + 1 = p' - i$.
- La partizione destra ha ampiezza $L_{\text{right}} = j - (p' + 1) + 1 = j - p'$.

La condizione del test $p' \leq \lfloor \frac{i+j}{2} \rfloor$ impone che:

$$p' - i \leq \frac{j - i}{2} < \frac{L}{2}. \quad (4.40)$$

Pertanto, se la condizione è verificata, la chiamata ricorsiva viene invocata sulla partizione sinistra, la cui dimensione è strettamente limitata da:

$$L_{\text{ricorsivo}} \leq \left\lfloor \frac{L}{2} \right\rfloor. \quad (4.41)$$

Nel caso complementare ($p' > \lfloor \frac{i+j}{2} \rfloor$), la partizione destra soddisfa analogamente $j - p' < \frac{L}{2}$, e la chiamata ricorsiva viene invocata sulla partizione destra, verificando la medesima disequazione.

La partizione rimanente possiede dimensione $L_{\text{iterativo}} \geq \frac{L}{2}$, ma essa **non genera alcuna chiamata ricorsiva**: l'algoritmo aggiorna semplicemente il puntatore $i \leftarrow p' + 1$ oppure $j \leftarrow p' - 1$ e itera nel corpo del ciclo ‘while’ senza allocare alcun nuovo stack frame.

Ne consegue che la dimensione della sotto-istanza associata a ciascuna chiamata ricorsiva nidificata sullo stack si riduce di almeno un fattore 2 rispetto al frame genitore:

$$L^{(k)} \leq \frac{n}{2^k}, \quad (4.42)$$

dove k rappresenta la profondità corrente dello stack. Il processo di ricorsione può proseguire al massimo fintanto che $L^{(k)} \geq m_0 \geq 1$:

$$\frac{n}{2^k} \geq 1 \implies 2^k \leq n \implies k \leq \lfloor \log_2 n \rfloor. \quad (4.43)$$

Poiché ciascun record di attivazione richiede una quantità costante di memoria $\mathcal{O}(1)$, il consumo totale di memoria ausiliaria è rigorosamente limitato a:

$$\text{Extra Space} = \mathcal{O}(1) \cdot \lfloor \log_2 n \rfloor = \Theta(\log_2 n). \quad (4.44)$$

□

4.4.5 Ottimizzazione Ibrida con Cutoff a Insertion Sort

L'introduzione della soglia $m_0 \in [16, 32]$ per il passaggio ad *Insertion Sort* risponde a un criterio fondamentale di *cache locality* ed efficienza esecutiva:

- **Eliminazione dell'overhead ricorsivo:** Quando la dimensione del sottoarray è inferiore a m_0 , il costo associato alla chiamata di funzione, al partizionamento e alla gestione dei puntatori supera il costo computazionale dei confronti diretti.
- **Efficienza di Insertion Sort sui piccoli blocchi:** *Insertion Sort* possiede un fattore costante di tempo eccezionalmente ridotto: l'algoritmo non necessita di memoria ausiliaria, esegue istruzioni elementari all'interno di un ciclo compatto e beneficia dell'inlining del codice da parte del compilatore.

- **Sfruttamento ottimale della Cache L1:** Un blocco di $16 \div 32$ elementi occupa circa $128 \div 256$ byte (per elementi da 64 bit), risiedendo integralmente all'interno di una o al massimo due linee di cache L1 (64 byte per linea). Di conseguenza, tutti i confronti e gli scambi avvengono senza generare alcun cache miss, massimizzando il throughput della CPU.
- **Comportamento su array quasi ordinati:** Poiché le macro-partizioni di Quicksort hanno già collocato gli elementi nelle vicinanze della loro posizione finale, l'esecuzione di Insertion Sort sui blocchi piccoli trova i dati in una configurazione "quasi ordinata", convergendo in un tempo strettamente lineare $\mathcal{O}(n)$ anziché quadratico.

In conclusione, Bounded Quicksort rappresenta una perfetta sintesi di rigore teorico e ingegneria algoritmica, garantendo la complessità temporale attesa ottima $\mathcal{O}(n \log_2 n)$, l'immunità da *Stack Overflow* grazie allo spazio extra $\Theta(\log_2 n)$, e la massima velocità empirica su hardware moderno.

Bibliografia

- [1] Donald E. Knuth. *The Art of Computer Programming*, volume 1. Addison-Wesley, Reading, MA, 3 edition, 1997.
- [2] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [3] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
- [4] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.
- [5] Vladimir Yaroslavskiy. Dual-pivot quicksort algorithm. *Research Disclosure*, 2009. Available at <http://iaroslavski.narod.ru/quicksort/DualPivotQuicksort.pdf>.